

Studienarbeit

Projektarbeit T3000

aus dem Studiengang Informationstechnik

an der Dualen Hochschule Baden-Württemberg Ravensburg Campus
Friedrichshafen

von

Finn Ludwig, Ekaterina Karavaeva, David Pelcz, Alexander
Bernecker

12.07.2026

Inhaltsverzeichnis

1. Einleitung	1
1.1. Motivation und Kontext	1
1.2. Problemstellung	2
1.3. Messbarkeit von Generalisierung	3
1.4. Offene Fragen und Forschungsfrage	4
1.5. Zielsetzung und Abgrenzung	5
1.6. Aufbau der Arbeit	7
2. Theoretische Grundlagen	7
2.1. Künstliche Intelligenz	7
2.2. Maschinelles Lernen und wichtige Methoden	7
2.2.1. Supervised Learning(überwachtes Lernen)	7
2.2.2. Unsupervised Learning (unüberwachtes Lernen)	8
2.2.3. Reinforcement Learning (bestärkendes Lernen)	8
2.2.4. Markov-Entscheidungsprozess	9
2.2.5. Policy und Value-Funktionen	9
2.2.6. On-Policy und Off-Policy	10
2.3. Proximal Policy Optimization (PPO)	11
2.3.1. Actor-Critic-Framework	11
2.3.2. Clipped Surrogate Objective	11
2.3.3. Generalized Advantage Estimation (GAE)	12
2.3.4. Trainingszyklus und Hyperparameter	12
2.4. Sequenzmodellierung für RL	13
2.4.1. Long Short-Term Memory (LSTM)	13
2.4.2. Transformer-Architektur	14
2.5. Wahrnehmung in RL-Agenten	16
2.5.1. Vektorbeobachtungen (VectorSensor)	16
2.5.2. Ray-Sensoren (RayPerceptionSensor)	16
2.5.3. Kamerasensor (CameraSensor)	17
2.5.4. Beobachtungsraum und Normalisierung	17
2.6. Unity ML-Agents Toolkit	17
2.6.1. Architektur	18
2.6.2. Komponenten	18
2.6.3. Trainings-Workflow	19
3. Stand der Technik	19
3.1. Simulationsumgebungen und das Unity-ML-Agents-Framework	20

3.2.	Lernbasierte Navigation in 3D-Umgebungen	20
3.2.1.	Anfänge der lernbasierten Navigationsforschung	20
3.2.2.	Grundlagen des Lernens unter partieller Beobachtbarkeit	21
3.2.3.	Grundlagen der Belohnungsgestaltung	22
3.2.4.	Generalisierung auf unbekannte Umgebungen	22
3.3.	Ableitung des Handlungsbedarfs für die vorliegende Arbeit	23
4.	Systemarchitektur und Umgebung	24
4.1.	Gesamtüberblick	24
4.2.	Map-System	25
4.2.1.	Datenmodell	25
4.2.2.	MapGenerator zur Laufzeit	26
4.2.3.	Prozedurale Generierung als Umsetzung der Messbarkeitsbedingung	27
4.3.	Agent-System	30
4.3.1.	Aktionsraum	31
4.3.2.	Observation-Space	31
4.3.3.	Bewegungs- und Sprungphysik	32
4.4.	Sensorik – und die begründete Wahl der Ray-Wahrnehmung	32
4.5.	Reward-System	35
4.6.	Trainingsinfrastruktur	36
5.	Methodik und experimentelles Design	38
5.1.	Wissenschaftliche Rahmung	38
5.2.	Vergleichsdesign	38
5.3.	Kontrollierte Variablen und begründete Abweichungen	38
5.3.1.	Identische Parameter	38
5.3.2.	Notwendige YAML-Abweichungen und ihre Begründung	39
5.4.	Abweichung in Tuning-Iterationen	40
5.4.1.	Vorgehensweise und Bewertungsgrundlage	41
5.4.2.	Iterationen der MLP-Baseline	41
5.5.	Evaluationsprotokoll	42
5.5.1.	Primärmetriken	42
5.5.2.	Generalisierungsmetriken	42
5.5.3.	Statistische Auswertung	42
6.	Modellarchitekturen	42
6.1.	MLP-Baseline	42
6.1.1.	Rolle der Baseline im Untersuchungsdesign	42
6.1.2.	Aufbau des Multilayer-Perzeptrons	43
6.1.3.	Erwartung an die Baseline	44
6.2.	Transformer-Memory (Custom Policy)	44
6.2.1.	Motivation: Memory unter partieller Beobachtbarkeit	44

6.2.2. Eingabeverarbeitung: Beobachtungssequenz und Embedding	44
6.2.3. Kern der Architektur: kausaler Self-Attention-Block	44
6.2.4. Sequenzaggregation und Actor-Critic-Anbindung	44
6.3. LSTM-Memory (Custom Policy)	44
7. Umsetzung	45
7.1. Aufbauphase (M1–M6)	45
7.1.1. Meilenstein 1: Map-System	45
7.1.2. Meilenstein 2: Mehrere Layouts	45
7.1.3. Meilenstein 3: Das Agent-Grundsystem	46
7.1.4. Meilenstein 4: Hindernisse, Todeslogik und Sensorik	46
7.1.5. Meilenstein 5: Reward-System und erstes Training	46
7.1.6. Meilenstein 6: Prozedurale Generierung und Curriculum	47
7.2. Transformer-Integration: von V1 bis zum lauffähigen Modell	48
7.2.1. Warum kein Fork: der venv-Patch-Ansatz	48
7.2.2. Warum kein Pre-Trained Modell	48
7.2.3. Problemklasse A - Framework-/Engine-Workarounds	48
7.2.4. Problemklasse B - Inference/Training-Konsistenz	48
7.2.5. Problemklasse C - Reward-/Curriculum-Pathologien	48
7.2.6. Iterationsübersicht V1-V22 (verdichtete Tabelle)	48
7.3. LSTM-Integration	48
7.4. MLP-Integration	49
7.4.1. Systemarchitektur der ML-Agents-Anbindung	50
7.4.2. Integration des Agenten in Unity	50
7.4.3. Trainingsszene und Parallelisierung	51
7.4.4. Trainingskonfiguration, Trainingsworkflow, Beobachtbarkeit und Inferenz	52
8. Evaluation	53
8.1. MLP-Baseline	53
8.2. Architektur-Vergleiche	53
8.3. Generalisierung auf held-out Maps	53
8.4. Statistische Auswertung	53
8.5. Diskussion	53
9. Übertragbarkeit und praktische Anwendbarkeit	53
9.1. Analogie Labyrinth	53
9.2. Hardware- und Software-Anforderungen	54
9.3. Bewertungsmatrix	54
9.4. Limitationen der Übertragbarkeit	54
10. Fazit und Ausblick	54
10.1. Zusammenfassung	54

10.2. Beantwortung der Forschungsfragen	54
10.3. Limitationen und Lessons Learned	54
10.4. Ausblick	54
11. Sec mlp baseline	54
Anhang	55
Literatur	56

Tabellenverzeichnis

Tabelle 1	Zuordnung zwischen realem Anwendungsszenario und Simulationsmodell	2
Tabelle 2	Software-Stack und Repository-Struktur des Gesamtsystems.	25
Tabelle 3	Datenmodell der Karten: Zelltyp-Satz und Felder von MapData.	26
Tabelle 4	Laufzeit-Kartenaufbau: Objektverwaltung, Platzierungslogik und Trainingsflächen je Szene.	27
Tabelle 5	Offline-Generierungspipeline: Einstiegspunkt, Konstanten und Lösbarkeitsregeln.	29
Tabelle 6	Schwierigkeitsleiter und trainiertes Curriculum: Enum-Reihenfolge, Phasenfolge und Konstruktionsparameter der Layout-Pools.	30
Tabelle 7	Behaviors des Agent-Systems: die drei Vergleichsagenten.	31
Tabelle 8	Diskreter Aktionsraum der drei Vergleichsagenten (drei Branches der Größe 3).	31
Tabelle 9	Bewegungs- und Sprungphysik des Agenten. jumpForce und maxUpwardVelocity sind die effektiven Werte aus Agent.prefab (Script-Defaults abweichend).	32
Tabelle 10	Vollständige Sensor- und Beobachtungsspezifikation der drei Vergleichsagenten (autoritative Quelle): 31 Vektor-Beobachtungen plus 176 Ray-Beobachtungen.	34
Tabelle 11	Belohnungsfunktion der drei Vergleichsagenten – identisch für alle Architekturen und vor den Vergleichsläufen eingefroren. Die Curiosity-Stärken je Behavior sind Teil der Trainer-Konfiguration und stehen in Tabelle 15.	36
Tabelle 12	Trainingsinfrastruktur: Umgebungen, Patch-Mechanismus und Konfigurationsebenen.	37
Tabelle 13	Operationalisierung der Forschungsfragen: Metrik, Hypothese, zuständiger Vergleich und Ergebnis-Abschnitt.	38

Tabelle 14 Identische Trainer-Parameter aller drei Behaviors im finalen Vergleichslauf.	39
Tabelle 15 Abweichungen der Trainer-Konfigurationen zwischen den Architekturen (model_comparison_final_v3.yaml). Alle übrigen Parameter sind identisch (Abschnitt 5.3.1); vollständige YAMLS im Anhang.	40

Abkürzungsverzeichnis

AI	Artificial Intelligence
BFS	Breadth-First Search
CNN	Convolutional Neural Network
GAE	Generalized Advantage Estimation
IT	Informationstechnologie
KI	Künstliche Intelligenz
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
ML	Machine Learning
MLP	Multi-Layer Perceptron
ONNX	Open Neural Network Exchange
PBRS	Potential-Based Reward Shaping
PPO	Proximal Policy Optimization
RL	Reinforcement Learning

1. Einleitung

1.1. Motivation und Kontext

Autonome Serviceroboter gewinnen in Dienstleistungsbranchen zunehmend an Bedeutung. In Bereichen wie Gastronomie, Hotellerie und Gesundheitswesen werden sie als Möglichkeit betrachtet, wiederkehrende Service- und Transportaufgaben zu automatisieren sowie Mitarbeitende bei alltäglichen Tätigkeiten zu entlasten. Die Fortschritte in den Bereichen Robotik, Sensorik und künstliche Intelligenz eröffnen dabei neue Möglichkeiten für den praktischen Einsatz intelligenter Systeme [1, S. 9 ff.].

Eine Voraussetzung für den erfolgreichen Einsatz solcher Systeme ist die Fähigkeit zur autonomen Navigation. Ein Serviceroboter muss seine Umgebung wahrnehmen, Hindernisse erkennen und selbstständig Entscheidungen treffen, um ein vorgegebenes Ziel sicher zu erreichen. Im Gegensatz zu klassischen regelbasierten Systemen sollen moderne Roboter dabei flexibel auf Veränderungen ihrer Umgebung reagieren können. Für die Entwicklung solcher Navigationsstrategien hat sich Reinforcement Learning (RL) als vielversprechender Ansatz etabliert. Agenten lernen durch Interaktion mit ihrer Umgebung eigenständig, welche Aktionen langfristig zu einer erfolgreichen Zielerreichung führen [2, S. 1 - 8].

Die Untersuchung entsprechender Lernverfahren auf realen Robotersystemen ist jedoch mit erheblichem Aufwand verbunden. Neben den Kosten für Hardware und Sensorik können Fehlentscheidungen während des Trainings zu Beschädigungen der Umgebung oder des Roboters führen. Darüber hinaus sind reale Experimente häufig zeitaufwendig und nur eingeschränkt reproduzierbar. Aus diesem Grund werden neue Verfahren zunächst in Simulationsumgebungen entwickelt und evaluiert [3, S. 1 - 6].

Um die für diese Arbeit relevante Navigationsaufgabe systematisch untersuchen zu können, wird die reale Umgebung eines Restaurants in ein abstrahiertes Simulationsmodell überführt. Ziel dieser Abstraktion ist nicht die möglichst realitätsnahe Nachbildung eines Restaurants, sondern die Reduktion auf jene Eigenschaften, die für die Navigation eines autonomen Agenten wesentlich sind. Der Agent muss weiterhin Wege finden, Hindernisse vermeiden und ein Ziel erreichen. Gleichzeitig wird die Umgebung so vereinfacht, dass unterschiedliche Lern-

verfahren unter identischen Bedingungen untersucht und miteinander verglichen werden können.

Die Beziehung zwischen realem Anwendungsszenario und Simulationsmodell ist in Tabelle 1 dargestellt.

Reale Umgebung	Simulationsmodell
Restaurantgänge und Laufwege	Korridore des Labyrinths
Tische, Stühle oder blockierte Bereiche	Hindernisse(Lava- und Hole-Felder)
Zielposition des Serviceroboters	Zielpattform
Unterschiedliche Restaurantgrundrisse	Variierende Kartenlayouts

Tabelle 1 – Zuordnung zwischen realem Anwendungsszenario und Simulationsmodell

Die Simulationsumgebung bildet damit die Grundlage für die kontrollierte Untersuchung unterschiedlicher Reinforcement-Learning-Ansätze für autonome Navigationsaufgaben.

1.2. Problemstellung

Die Fähigkeit eines Agenten, eine einzelne Trainingsumgebung erfolgreich zu durchlaufen, stellt noch keinen ausreichenden Nachweis für intelligentes Navigationsverhalten dar. Hohe Erfolgsraten können auch dann erreicht werden, wenn der Agent lediglich spezifische Kartenstrukturen oder Bewegungsabfolgen auswendig lernt. In diesem Fall wäre das erlernte Verhalten eng an die Trainingsumgebung gebunden und nur eingeschränkt auf neue Situationen übertragbar.

Für reale Anwendungen ist ein solches Verhalten jedoch unzureichend. Restaurants unterscheiden sich hinsichtlich ihrer Raumaufteilung, der Anordnung von Tischen und Laufwegen sowie möglicher Hindernisse. Darüber hinaus können sich diese Gegebenheiten während des Betriebs verändern. Ein autonomer Serviceroboter muss daher in der Lage sein, bereits erlernte Strategien auf unbekannte Umgebungen anzuwenden, anstatt ausschließlich bekannte Situationen wiederzuerkennen.

Diese Fähigkeit wird als Generalisierung bezeichnet. Im Kontext des Reinforcement Learnings beschreibt Generalisierung die Fähigkeit eines Agenten, auch in Umgebungen erfolgreich zu agieren, die während des Trainings nicht beobachtet wurden. Sie stellt eine zentrale Herausforderung moderner Lernverfahren dar, da eine gute Leistung auf Trainingsdaten nicht automatisch eine gute Leistung auf unbekannten Situationen garantiert [4, S. 111 - 120].

Neben der Generalisierung spielt die Verarbeitung zeitlicher Zusammenhänge eine wichtige Rolle. Während sich einige Navigationsentscheidungen allein auf Basis der aktuellen Wahrnehmung treffen lassen, existieren auch Situationen, in denen Informationen aus vergangenen Beobachtungen relevant werden.

Klassische Feed-Forward-Netzwerke besitzen kein explizites Gedächtnis und treffen ihre Entscheidungen ausschließlich auf Grundlage der aktuellen Eingabe. Architekturansätze wie Long Short-Term Memory Networks (LSTM) oder Transformer-Modelle ermöglichen dagegen die Berücksichtigung vergangener Beobachtungen und könnten dadurch Vorteile bei komplexen Navigationsaufgaben bieten.

Vor diesem Hintergrund stellt sich die Frage, welchen Einfluss unterschiedliche Netzwerkarchitekturen auf die Fähigkeit eines Reinforcement-Learning-Agenten haben, robuste und übertragbare Navigationsstrategien zu erlernen.

1.3. Messbarkeit von Generalisierung

Da Generalisierung das zentrale Qualitätskriterium dieser Arbeit darstellt, muss ihre Bewertung objektiv und reproduzierbar erfolgen. Um Generalisierung nachweisen zu können, muss die Leistung eines Agenten auf Umgebungen untersucht werden, die während des Trainings nicht beobachtet wurden. Nur unter dieser Voraussetzung lässt sich feststellen, dass das erlernte Verhalten auf neue Situationen übertragbar ist.

Aus dieser Anforderung ergibt sich unmittelbar die Notwendigkeit einer Trennung zwischen Trainings- und Testumgebungen. Während Trainingskarten dem Agenten zum Lernen zur Verfügung stehen, dürfen Testkarten während des Trainingsprozesses niemals verwendet werden. Die Generalisierungsfähigkeit eines Agenten kann anschließend anhand seiner Leistung auf diesen zuvor ungesehenen Karten bewertet werden.

Die Erstellung einer ausreichend großen Anzahl voneinander unabhängiger Karten stellt dabei eine weitere Herausforderung dar. Eine manuelle Konstruktion wäre mit erheblichem Aufwand verbunden und würde die Vielfalt möglicher Umgebungen einschränken. Aus diesem Grund wird in dieser Arbeit eine prozedurale Kartengenerierung eingesetzt. Diese ermöglicht die automatische Erzeugung unterschiedlicher Kartenlayouts und damit die systematische Trennung von Trainings- und Testumgebungen.

Die prozedurale Kartengenerierung wird in dieser Arbeit daher nicht als eigenständiges Forschungsziel betrachtet. Sie stellt vielmehr eine notwendige Voraussetzung dar, um die

Generalisierungsfähigkeit der untersuchten Agenten objektiv und reproduzierbar messen zu können.

1.4. Offene Fragen und Forschungsfrage

Aus der dargestellten Problemstellung ergeben sich mehrere offene Fragestellungen. Zunächst muss untersucht werden, wie Reinforcement-Learning-Agenten innerhalb einer dreidimensionalen Simulationsumgebung implementiert werden können und welche Anforderungen sich daraus für die Gestaltung der Umgebung ergeben.

Darüber hinaus stellt sich die Frage, auf welcher Informationsgrundlage der Agent seine Entscheidungen trifft. Damit ein Agent zielgerichtet handeln kann, müssen relevante Eigenschaften der Umgebung erfasst und in einer für das neuronale Netzwerk verarbeitbaren Form bereitgestellt werden. Die konkrete Ausgestaltung der Wahrnehmung beeinflusst maßgeblich, welche Informationen dem Agenten während der Navigation überhaupt zur Verfügung stehen.

Eine weitere offene Fragestellung betrifft die Verarbeitung dieser Informationen. Während einige Navigationssituationen allein anhand der aktuellen Beobachtung lösbar sind, können in komplexeren Labyrinthstrukturen Situationen entstehen, in denen zusätzliche Kontextinformationen benötigt werden. Beispielsweise kann ein Agent in Sackgassen oder an mehrdeutigen Kreuzungen davon profitieren, Informationen aus vergangenen Beobachtungen in seine Entscheidungsfindung einzubeziehen.

Klassische Feed-Forward-Netzwerke treffen ihre Entscheidungen ausschließlich auf Basis der aktuellen Eingabe und verfügen über kein explizites Gedächtnis. Demgegenüber ermöglichen Architekturen wie Long Short-Term Memory Networks (LSTM) oder Transformer-Modelle die Berücksichtigung zeitlicher Zusammenhänge über mehrere Zeitschritte hinweg. Daraus ergibt sich die Vermutung, dass Gedächtnismechanismen insbesondere bei Navigationsaufgaben in partiell beobachtbaren Umgebungen einen Vorteil bieten könnten.

Vor diesem Hintergrund ergibt sich die zentrale Forschungsfrage dieser Arbeit:

„Welche neuronale Netzwerkarchitektur (MLP, LSTM oder Transformer) ist für einen Reinforcement-Learning-Agenten geeignet, um in einer prozedural generierten 3D-Labyrinthwelt generalisierbares Navigations- und Hindernisvermeidungsverhalten zu erlernen, das sich auf unbekannte Kartenlayouts übertragen lässt?“

Zur Beantwortung dieser Forschungsfrage werden die folgenden Teilfragen untersucht:

F1: Mehrwert temporalen Gedächtnisses

Verbessert ein temporaler Gedächtnismechanismus in Form eines LSTM- oder Transformer-Modells die Navigationsleistung eines Reinforcement-Learning-Agenten gegenüber einem gedächtnislosen MLP-Basisagenten?

F2: Vergleich von LSTM und Transformer

Welche Unterschiede zeigen LSTM- und Transformer-Architekturen hinsichtlich Lernverhalten, Trainingsdynamik und Leistungsfähigkeit bei der Ray-basierten Navigation in einer Labyrinthumgebung?

F3: Generalisierung auf unbekannte Kartenlayouts

Welche der untersuchten Architekturen (MLP, LSTM oder Transformer) generalisiert am zuverlässigsten auf prozedural generierte Karten, die während des Trainings nicht beobachtet wurden?

Die Teilfragen bauen logisch aufeinander auf. Zunächst wird untersucht, ob ein Gedächtnismechanismus für die betrachtete Navigationsaufgabe grundsätzlich einen Mehrwert bietet. Anschließend erfolgt ein direkter Vergleich der beiden Gedächtnisarchitekturen LSTM und Transformer. Abschließend wird analysiert, in welchem Umfang sich die erlernten Strategien auf bislang unbekannte Kartenlayouts übertragen lassen.

Zur Beantwortung der Forschungsfragen werden die folgenden Hypothesen aufgestellt:

H1: Agenten mit temporalem Gedächtnis (LSTM oder Transformer) erreichen eine höhere Navigationsleistung als ein gedächtnisloser MLP-Basisagent.

H2: Transformer-Architekturen erzielen eine höhere Endleistung als LSTM-Modelle, benötigen jedoch mehr Trainingsaufwand bis zur Konvergenz.

H3: Architekturen mit temporalem Gedächtnis weisen eine höhere Generalisierungsfähigkeit auf als ein MLP-Basisagent und zeigen ein geringeres Maß an Overfitting gegenüber den Trainingskarten.

Die Operationalisierung dieser Forschungsfragen sowie die zur Evaluation verwendeten Metriken und Auswertungsverfahren werden in Kapitel 2 beschrieben

1.5. Zielsetzung und Abgrenzung

Ziel dieser Arbeit ist die Entwicklung und Evaluation einer Reinforcement-Learning-Umgebung zur Untersuchung generalisierbarer Navigationsstrategien in dreidimensionalen

Labyrinthwelten. Hierzu wird auf Basis der Unity-Spielengine und des ML-Agents Toolkits eine Simulationsumgebung entwickelt, in der autonome Agenten lernen, ein Ziel zu erreichen und gleichzeitig verschiedene Hindernisse zu vermeiden.

Im Mittelpunkt der Untersuchung steht der Vergleich dreier unterschiedlicher neuronaler Netzwerkarchitekturen. Neben einem Multi-Layer-Perceptron (MLP) als gedächtnislose Baseline werden ein Long Short-Term Memory Network (LSTM) sowie ein Transformer-Modell implementiert und unter vergleichbaren Bedingungen trainiert. Die Leistungsfähigkeit der Architekturen wird anhand ihrer Navigationsleistung sowie ihrer Fähigkeit zur Generalisierung auf zuvor ungesehene Kartenlayouts bewertet.

Die entwickelte Umgebung umfasst eine Ray-basierte Wahrnehmung, verschiedene Hindernistypen wie Lavafelder, Löcher und Sackgassen sowie ein System zur prozeduralen Kartengenerierung. Durch die automatische Erzeugung unterschiedlicher Kartenlayouts können Trainings- und Testumgebungen voneinander getrennt werden, wodurch eine systematische Bewertung der Generalisierungsfähigkeit ermöglicht wird.

Zum Pflichtumfang der Arbeit gehören:

- die Entwicklung einer dreidimensionalen Labyrinthumgebung in Unity,
- die Implementierung eines Reinforcement-Learning-Agenten auf Basis von PPO,
- die Integration und Evaluation der Architekturen MLP, LSTM und Transformer,
- die Implementierung einer Ray-basierten Wahrnehmung,
- die Durchführung von Generalisierungstests auf zuvor ungesehenen Karten,
- die Dokumentation der Ergebnisse sowie die Bereitstellung einer reproduzierbaren Trainings- und Evaluationsumgebung.

Darüber hinaus werden mehrere Erweiterungen umgesetzt. Hierzu zählen insbesondere die prozedurale Kartengenerierung, Curriculum Learning zur schrittweisen Steigerung der Aufgabenschwierigkeit, Multi-Area-Training zur effizienteren Datensammlung sowie der direkte Vergleich unterschiedlicher Gedächtnisarchitekturen.

Von der Zielsetzung bewusst abgegrenzt werden Fragestellungen, die den Umfang dieser Arbeit überschreiten würden. Hierzu zählen insbesondere der Transfer der Ergebnisse auf reale Robotersysteme (Sim-to-Real), Mehragentenszenarien, dynamische Hindernisse, die Untersuchung unterschiedlicher Sensormodalitäten sowie die Entwicklung neuer Reinforcement-Learning-Algorithmen. Ebenso erfolgt keine quantitative Bewertung der Übertragbarkeit auf reale Serviceroboter. Diese wird stattdessen in Kapitel 9 qualitativ diskutiert.

Der Fokus der Arbeit liegt somit ausschließlich auf der Frage, welchen Einfluss unterschiedliche neuronale Netzwerkarchitekturen auf das Erlernen und die Generalisierung von Navigationsverhalten in einer kontrollierten Simulationsumgebung besitzen.

1.6. Aufbau der Arbeit

2. Theoretische Grundlagen

2.1. Künstliche Intelligenz

Künstliche Intelligenz (KI) bezeichnet den Bereich der Informatik, der sich mit der Entwicklung maschinenbasierter Systeme befasst, die Aufgaben ausführen können, die menschliche Intelligenz erfordern, wie beispielsweise Problemlösung, Sprachverständnis oder Mustererkennung [5, S. 276]

2.2. Maschinelles Lernen und wichtige Methoden

Maschinelles Lernen (ML) ist ein Teilgebiet der KI und befasst sich mit der Entwicklung von Algorithmen, die aus Daten lernen und auf Grundlage dieser Daten Vorhersagen treffen [5, S. 276 f.]. Dabei müssen keine Entscheidungsregeln explizit programmiert werden, da diese während des Trainingsprozesses aus den vorhandenen Daten abgeleitet werden. Abhängig davon, welche Art von Daten dem Modell zur Verfügung stehen und wie der Lernprozess organisiert ist, lassen sich verschiedene Lernparadigmen unterscheiden. Zu den grundlegenden Ansätzen zählen Supervised Learning (überwachtes Lernen), Unsupervised Learning (unüberwachtes Lernen) und Reinforcement Learning (bestärkendes Lernen) [6, S. 282 f.].

2.2.1. Supervised Learning(überwachtes Lernen)

Beim überwachten Lernen (Supervised Learning) wird ein Modell anhand von annotierten Trainingsdaten trainiert. Jeder Eingabe x_i ist dabei ein bekanntes Ziel y_i zugeordnet. Das Ziel des Lernprozesses besteht darin, eine Funktion $f : X \rightarrow Y$ zu erlernen, die für neue, unbekannte Eingaben möglichst korrekte Vorhersagen liefert. Typische Anwendungsgebiete sind die Klassifikation, bei der Daten vordefinierten Klassen zugeordnet werden, sowie die

Regression, bei der kontinuierliche Werte vorhergesagt werden [6, S. 282]. Der Lernfortschritt wird durch den Vergleich der Modellausgabe mit den bekannten Zielwerten bewertet und über geeignete Optimierungsverfahren verbessert.

2.2.2. Unsupervised Learning (unüberwachtes Lernen)

Im Gegensatz dazu stehen beim unüberwachten Lernen (Unsupervised Learning) keine Zielwerte oder Labels zur Verfügung. Das Modell erhält ausschließlich die Eingabedaten und versucht selbstständig, darin enthaltene Strukturen, Zusammenhänge oder Muster zu identifizieren. Häufige Verfahren sind das Clustering, bei dem ähnliche Datenpunkte zu Gruppen zusammengefasst werden, sowie die Dimensionsreduktion, die darauf abzielt, die wesentlichen Informationen eines Datensatzes in einer kompakteren Darstellung abzubilden [6, S. 283]. Unüberwachtes Lernen eignet sich insbesondere zur explorativen Datenanalyse und zur Entdeckung bisher unbekannter Strukturen.

2.2.3. Reinforcement Learning (bestärkendes Lernen)

Das für diese Arbeit zentral relevante Verfahren ist das bestärkende Lernen (Reinforcement Learning, RL). Dieses verfolgt einen anderen Ansatz. Hier interagiert ein Agent fortlaufend mit einer Umgebung und trifft Entscheidungen in Form von Aktionen. Für diese Aktionen erhält er ein Feedback in Form eines sogenannten Reward-Signals. Anders als beim überwachten Lernen werden dem Agenten keine direkten Korrekturen für einzelne Entscheidungen gegeben. Stattdessen muss er durch Versuch und Irrtum selbst erlernen, welche Handlungsstrategien langfristig zu einem möglichst hohen kumulativen Reward führen [2, S. 1 f.]. Aufgrund dieses Ansatzes eignet sich Reinforcement Learning besonders für komplexe Entscheidungsprobleme mit zeitlicher Abhängigkeit, beispielsweise in der Robotik, Navigation oder bei der Entwicklung von Spielstrategien. Eine besondere Herausforderung im bestärkenden Lernen stellt das sogenannte Exploration-Exploitation-Dilemma dar. Der Agent muss kontinuierlich abwägen, ob er bereits bekannte und erfolgversprechende Aktionen ausführt (Exploitation) oder neue, bislang wenig erforschte Aktionen ausprobiert (Exploration), die langfristig zu einer höheren Belohnung führen könnten [2, S. 3]. Die Balance zwischen diesen beiden Strategien ist ein zentraler Aspekt vieler Reinforcement-Learning-Verfahren und hat entscheidenden Einfluss auf deren Leistungsfähigkeit.

2.2.4. Markov-Entscheidungsprozess

Der theoretische Rahmen des bestärkenden Lernens wird durch den Markov-Entscheidungsprozess (Markov Decision Process, MDP) beschrieben. Ein MDP modelliert die Interaktion eines Agenten mit seiner Umgebung und bildet damit eine zentrale Grundlage vieler Reinforcement-Learning-Verfahren. Dabei befindet sich der Agent zu einem Zeitpunkt in einem Zustand, wählt eine Aktion aus, erhält anschließend eine Belohnung und gelangt in einen Folgezustand [2, S. 47 f.].

Ein MDP umfasst dabei insbesondere einen Zustandsraum S , einen Aktionsraum A , Übergangswahrscheinlichkeiten, eine Rewardfunktion sowie einen Diskontierungsfaktor γ . Der Zustandsraum S enthält die möglichen Zustände der Umgebung. Der Aktionsraum A beschreibt die Aktionen, die dem Agenten zur Verfügung stehen. Die Übergangswahrscheinlichkeit $p(s' | s, a)$ gibt an, mit welcher Wahrscheinlichkeit der Agent nach Ausführung der Aktion a im Zustand s in den Folgezustand s' übergeht.

Die Rewardfunktion beschreibt die unmittelbare Rückmeldung, die der Agent nach einer Zustandsänderung erhält. In dieser Arbeit wird sie als erwartete unmittelbare Belohnung eines Übergangs verstanden. Sie kann daher abhängig vom aktuellen Zustand s , der ausgeführten Aktion a und dem Folgezustand s' betrachtet werden. Der Diskontierungsfaktor γ bestimmt, wie stark zukünftige Belohnungen gegenüber unmittelbar erhaltenen Belohnungen gewichtet werden. Werte nahe 0 führen dazu, dass kurzfristige Belohnungen stärker berücksichtigt werden, während Werte nahe 1 langfristige Strategien begünstigen.

Eine zentrale Annahme des MDP ist die Markov-Eigenschaft. Sie besagt, dass die zukünftige Entwicklung des Systems durch den aktuellen Zustand und die gewählte Aktion hinreichend beschrieben wird. Frühere Zustände und Aktionen liefern keine zusätzlichen Informationen, sofern der aktuelle Zustand alle für die Zukunft relevanten Informationen enthält. Durch diese Annahme lassen sich sequenzielle Entscheidungsprobleme strukturiert modellieren und für Reinforcement-Learning-Verfahren nutzbar machen [2, S.48 ff.].

2.2.5. Policy und Value-Funktionen

Das zentrale Ziel eines Reinforcement-Learning-Agenten besteht darin, eine möglichst gute Policy zu erlernen. Eine Policy beschreibt das Verhalten des Agenten, indem sie festlegt, welche Aktion in einem bestimmten Zustand ausgewählt wird. Bei einer stochastischen Policy gibt $\pi(a | s)$ die Wahrscheinlichkeit an, im Zustand s die Aktion a auszuwählen [2, S. 58]. Ziel des Lernprozesses ist es, eine Policy zu finden, die über die Zeit einen möglichst hohen

erwarteten Return erzielt. Der Return G_t beschreibt die diskontierte Summe der zukünftigen Rewards, die ein Agent nach dem Zeitpunkt t erhält. Formal ergibt sich der diskontierte Return zu:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Dabei bezeichnet R_{t+k+1} den Reward zu einem zukünftigen Zeitpunkt und γ den Diskontierungsfaktor. Durch die Diskontierung haben unmittelbar eintretende Rewards einen stärkeren Einfluss auf den Return als weiter in der Zukunft liegende Rewards. Der Agent wird dadurch dazu angehalten, nicht nur kurzfristige Belohnungen, sondern auch langfristige Konsequenzen seiner Handlungen zu berücksichtigen [2, S. 58 f.].

Um Zustände und Aktionen hinsichtlich ihres erwarteten langfristigen Nutzens bewerten zu können, werden im Reinforcement Learning sogenannte Value-Funktionen verwendet. Die State-Value-Funktion $v_{\pi}(s)$ beschreibt den erwarteten Return, den ein Agent erzielt, wenn er sich im Zustand s befindet und anschließend der Policy π folgt:

$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$ Die State-Value-Funktion bewertet somit Zustände. Sie gibt an, wie vorteilhaft es langfristig ist, sich unter einer bestimmten Policy in einem bestimmten Zustand zu befinden.

Neben der Bewertung von Zuständen ist häufig auch die Bewertung konkreter Aktionen relevant. Die Action-Value-Funktion $q_{\pi(s,a)}$ beschreibt den erwarteten Return, wenn der Agent im Zustand s zunächst die Aktion a ausführt und anschließend der Policy π folgt:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Damit erlaubt die Action-Value-Funktion eine detailliertere Bewertung möglicher Handlungsalternativen. Sie ist insbesondere für Verfahren relevant, bei denen Aktionen anhand ihres erwarteten langfristigen Nutzens verglichen werden, beispielsweise bei Q-Learning oder Deep Q-Networks. In solchen Verfahren werden Action-Value-Funktionen genutzt oder approximiert, um in einem Zustand geeignete Aktionen auszuwählen [2, S. 58-68].

2.2.6. On-Policy und Off-Policy

Reinforcement-Learning-Verfahren lassen sich unter anderem danach unterscheiden, ob sie on-policy oder off-policy lernen. Bei On-Policy-Verfahren wird die Policy verbessert, die auch zur Erzeugung der Trainingsdaten verwendet wird. Der Agent lernt somit aus Erfahrungen, die mit seiner aktuellen oder nur leicht veränderten Policy gesammelt wurden. Ein klassisches Beispiel für ein On-Policy-Verfahren ist Sarsa [2, S. 129].

Bei Off-Policy-Verfahren wird hingegen zwischen einer Ziel-Policy und einer Verhaltens-Policy unterschieden. Die Verhaltens-Policy erzeugt die Trainingsdaten, während die Ziel-Policy gelernt oder verbessert wird. Dadurch können Off-Policy-Verfahren auch aus Erfahrungen lernen, die nicht direkt durch die aktuell zu optimierende Policy erzeugt wurden. Ein klassisches Beispiel hierfür ist Q-Learning [2, S. 131].

Für diese Arbeit ist die Unterscheidung relevant, da Proximal Policy Optimization (PPO) zu den On-Policy-Verfahren zählt. Die Trainingsdaten werden daher mit der aktuellen Policy des Agenten gesammelt und anschließend zur schrittweisen Verbesserung dieser Policy genutzt.

2.3. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) ist ein Policy-Gradient-Verfahren, das zwischen der Datenerhebung durch Interaktion mit der Umgebung und der Optimierung einer sogenannten Surrogate-Zielfunktion wechselt. Im Gegensatz zu einfachen Policy-Gradient-Verfahren erlaubt PPO mehrere Optimierungsschritte auf denselben gesammelten Rollout-Daten, ohne dass die Policy dabei zu stark verändert werden soll [7, S. 1].

2.3.1. Actor-Critic-Framework

PPO wird in der Praxis häufig im Actor-Critic-Framework eingesetzt. Dabei übernimmt der Actor die Repräsentation der Policy $\pi_{\theta}(a | s)$ und bestimmt, mit welcher Wahrscheinlichkeit eine Aktion a in einem Zustand s ausgewählt wird. Der Critic schätzt den langfristigen Nutzen eines Zustands über eine Value-Funktion und dient damit als Bewertungsinstanz für die vom Actor gewählten Aktionen. Ein zentrales Konzept hierbei ist der Advantage. Der Advantage beschreibt, ob eine ausgeführte Aktion besser oder schlechter war als vom Critic erwartet. Ist der Advantage positiv, spricht dies dafür, dass die gewählte Aktion in einer vergleichbaren Situation wahrscheinlicher werden sollte. Ist der Advantage negativ, sollte ihre Wahrscheinlichkeit entsprechend reduziert werden. Durch diese Trennung zwischen Actor und Critic kann die Varianz der Policy-Gradient-Schätzung reduziert und das Training stabilisiert werden [2, S. 331 - 336].

2.3.2. Clipped Surrogate Objective

Das zentrale Problem klassischer Policy-Gradient-Methoden besteht darin, dass zu große Aktualisierungsschritte die Policy stark verschlechtern können. PPO adressiert dieses Problem durch ein Clipping-Verfahren, das die Wirkung zu großer Änderungen im Policy-Update

begrenzt [7, S. 3 f.]. Dazu wird zunächst das Wahrscheinlichkeitsverhältnis zwischen neuer und alter Policy für die tatsächlich ausgeführte Aktion definiert:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

Dieses Verhältnis beschreibt, wie stark sich die Wahrscheinlichkeit der gewählten Aktion unter der neuen Policy im Vergleich zur alten Policy verändert hat. Ein Wert von $r_t(\theta) = 1$ bedeutet, dass beide Policies die Aktion gleich wahrscheinlich bewerten. Werte größer als 1 bedeuten, dass die neue Policy die Aktion wahrscheinlicher macht; Werte kleiner als 1 bedeuten, dass sie die Aktion weniger wahrscheinlich macht. Die zentrale PPO-Zielfunktion mit Clipping lautet:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

Dabei bezeichnet $\hat{A}_t$ den geschätzten Advantage zum Zeitpunkt t . Der erste Term entspricht dem ungeclippten Policy-Gradient-Ziel. Der zweite Term begrenzt das Wahrscheinlichkeitsverhältnis auf den Bereich $[1 - \varepsilon, 1 + \varepsilon]$. Durch den Minimum-Operator wird verhindert, dass Änderungen der Policy, die zu einer zu starken Verbesserung des Zielfunktionswertes führen würden, unbeschränkt ausgenutzt werden. Dadurch wirkt PPO großen und potenziell destruktiven Policy-Updates entgegen [7, S. 3 f.]. Ein häufig verwendeter Wert ist $\varepsilon = 0.2$. Dies bedeutet jedoch nicht, dass sich die gesamte Policy maximal um 20 % ändern darf. Korrekt ist, dass das Wahrscheinlichkeitsverhältnis im geclippten Term auf den Bereich $[0.8, 1.2]$ begrenzt wird [7, S. 3 f.].

2.3.3. Generalized Advantage Estimation (GAE)

Für die praktische Schätzung des Advantage wird in PPO häufig Generalized Advantage Estimation (GAE) verwendet. GAE kombiniert Informationen aus mehreren Zeitschritten und steuert über den Parameter λ den Trade-off zwischen Bias und Varianz der Advantage-Schätzung. Kleine Werte von λ reduzieren typischerweise die Varianz, können aber stärkeren Bias verursachen. Große Werte verringern den Bias, erhöhen jedoch meist die Varianz. In vielen PPO-Konfigurationen wird $\lambda = 0.95$ verwendet [7, S. 10], [8, S. 1 f.].

2.3.4. Trainingszyklus und Hyperparameter

Ein PPO-Trainingszyklus besteht typischerweise aus mehreren Schritten. Zunächst sammelt der Agent mit seiner aktuellen Policy Rollouts in der Umgebung. Anschließend werden

Advantage-Schätzungen berechnet. Danach wird die Policy über mehrere Epochen mit Minibatches der gesammelten Daten aktualisiert. Nach Abschluss der Optimierung wird die aktualisierte Policy zur neuen alten Policy für die nächste Datensammlung [7, S. 4 f.].

Für diese Arbeit ist PPO besonders geeignet, da es als On-Policy-Verfahren gut zu kontrollierten Simulationsumgebungen passt und im Unity ML-Agents Framework als etablierter Trainingsalgorithmus verfügbar ist. Gegenüber DQN, das wertbasiert und off-policy arbeitet, sowie SAC, das insbesondere für kontinuierliche Aktionsräume verbreitet ist, stellt PPO für die untersuchte Navigationsaufgabe eine robuste und praktikable Wahl dar.

2.4. Sequenzmodellierung für RL

Der MDP setzt durch die Markov-Eigenschaft voraus, dass der aktuelle Zustand alle für zukünftige Entscheidungen relevanten Informationen enthält. In praktischen Anwendungen ist diese Voraussetzung aus Sicht des Agenten jedoch häufig nicht vollständig erfüllt, da der Agent meist nur eine begrenzte Beobachtung der Umgebung erhält. Sutton und Barto betonen, dass eine Zustandsrepräsentation nicht auf unmittelbare Sensordaten beschränkt sein muss, sondern auch aus vergangenen Wahrnehmungen oder einem internen Gedächtnis aufgebaut werden kann [2, S. 49].

Bei einem Agenten mit Ray-Sensoren beschreibt ein einzelner Timestep beispielsweise nur einen lokalen Ausschnitt der Umgebung. Dadurch kann der Agent aus einer einzelnen Beobachtung nicht zuverlässig ableiten, ob er sich bereits in einer Sackgasse befindet oder welchen Weg er zuvor genommen hat. Solche Problemstellungen lassen sich als partiell beobachtbare Entscheidungsprobleme auffassen, bei denen vergangene Beobachtungen und Aktionen zusätzliche Informationen für die Entscheidungsfindung liefern können.

Sequenzmodelle wie LSTM oder Transformer können diese zeitlichen Informationen nutzen, indem sie mehrere vergangene Zeitschritte berücksichtigen. Dadurch kann der Agent eine bessere interne Repräsentation seiner aktuellen Situation aufbauen und fundiertere Entscheidungen treffen.

2.4.1. Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) ist eine spezielle Variante rekurrenter neuronaler Netze (Recurrent Neural Networks, RNNs), die entwickelt wurde, um Informationen über lange Zeiträume hinweg speichern und verarbeiten zu können [9, S. 404]. Während klassische RNNs grundsätzlich für die Verarbeitung sequenzieller Daten geeignet sind, stoßen sie

bei langen Eingabesequenzen häufig an ihre Grenzen. Ursache hierfür ist das sogenannte Vanishing-Gradient-Problem, bei dem die während des Trainings berechneten Gradienten mit zunehmender Sequenzlänge immer kleiner werden. Dadurch wird es für das Netzwerk schwierig, Abhängigkeiten zwischen weit auseinanderliegenden Zeitschritten zu erlernen und relevante Informationen langfristig zu speichern [9, S. 404]. Um dieses Problem zu lösen, erweitert LSTM die Architektur klassischer RNNs um eine Speicherstruktur, die als Zellzustand bezeichnet wird. Dieser dient als internes Gedächtnis und ermöglicht die Weitergabe wichtiger Informationen über viele Zeitschritte hinweg. Der Informationsfluss innerhalb des Netzwerks wird dabei durch mehrere lernbare Steuermechanismen, sogenannte Gates, kontrolliert [9, S. 404 ff.]. Das Forget Gate entscheidet, welche Informationen aus dem bisherigen Gedächtnis beibehalten und welche verworfen werden. Dadurch kann das Netzwerk nicht mehr relevante Informationen gezielt vergessen. Das Input Gate bestimmt, welche neuen Informationen aus dem aktuellen Eingabeschritt in den Zellzustand aufgenommen werden. Das Output Gate legt schließlich fest, welche Teile des internen Gedächtnisses als Ausgabe an den nächsten Zeitschritt beziehungsweise an nachfolgende Netzwerkschichten weitergegeben werden [9, S. 406 f.]. Durch das Zusammenspiel dieser Gates kann ein LSTM relevante Informationen über viele Zeitschritte hinweg speichern und gleichzeitig irrelevante Informationen verwerfen. Dadurch eignet sich die Architektur besonders für Aufgaben, bei denen zeitliche Abhängigkeiten eine wichtige Rolle spielen, beispielsweise bei der Sprachverarbeitung, Zeitreihenanalyse oder im Reinforcement Learning. Das Gedächtnis eines LSTM wird durch einen kontinuierlich aktualisierten internen Zustand repräsentiert. Welche Informationen gespeichert, überschrieben oder vergessen werden, wird nicht manuell festgelegt, sondern während des Trainings automatisch erlernt. Im Kontext von Reinforcement Learning ermöglicht dies dem Agenten, Informationen aus vergangenen Beobachtungen zu berücksichtigen und dadurch fundiertere Entscheidungen zu treffen. In Unity ML-Agents ist LSTM bereits integriert und kann über die Konfigurationsoption `use_recurrent: true` aktiviert werden. Dadurch erhält der Agent eine Form von Gedächtnis, die es ihm erlaubt, auch in teilweise beobachtbaren Umgebungen historische Informationen in seine Entscheidungsfindung einzubeziehen.

2.4.2. Transformer-Architektur

Die Transformer-Architektur wurde von Vaswani et al. im Jahr 2017 in der einflussreichen Arbeit „Attention Is All You Need“ vorgestellt und hat die Verarbeitung sequenzieller Daten grundlegend verändert [10]. Ursprünglich wurde sie für Anwendungen der natürlichen

Sprachverarbeitung entwickelt, findet heute jedoch auch in zahlreichen anderen Bereichen des maschinellen Lernens Anwendung. Im Gegensatz zu rekurrenten Architekturen wie LSTM verarbeitet ein Transformer eine Sequenz nicht schrittweise, sondern betrachtet alle Elemente eines definierten Sequenzfensters gleichzeitig. Dadurch können Abhängigkeiten zwischen verschiedenen Positionen einer Sequenz parallel analysiert werden, was sowohl die Trainingsgeschwindigkeit erhöht als auch die Modellierung weitreichender Zusammenhänge erleichtert. Das zentrale Element der Architektur ist der sogenannte Self-Attention-Mechanismus. Dieser ermöglicht es dem Modell, die Relevanz einzelner Sequenzelemente für die Verarbeitung eines bestimmten Zeitschritts zu bewerten. Jedes Element einer Sequenz kann somit direkt Informationen von allen anderen Elementen berücksichtigen, unabhängig davon, wie weit diese zeitlich voneinander entfernt sind. Auf diese Weise lassen sich langfristige Abhängigkeiten erfassen, ohne die Einschränkungen rekurrenter Strukturen in Kauf nehmen zu müssen [10, S. 6 f.]. Zur weiteren Steigerung der Modellkapazität wird häufig Multi-Head-Attention eingesetzt. Dabei werden mehrere Attention-Mechanismen parallel ausgeführt, sodass unterschiedliche Beziehungen und Muster innerhalb derselben Sequenz gleichzeitig erlernt werden können. Die einzelnen Aufmerksamkeitsköpfe fokussieren sich dabei auf verschiedene Aspekte der Eingabedaten und tragen gemeinsam zu einer umfassenderen Repräsentation der Sequenz bei [10, S. 4 f.]. Da die Transformer-Architektur keine rekurrenten Verbindungen besitzt und somit keine inhärente Kenntnis über die Reihenfolge der Eingabedaten hat, muss die Positionsinformation explizit bereitgestellt werden. Dies geschieht durch sogenannte Positional Encodings, welche die Position jedes Sequenzelements kodieren und dem Modell ermöglichen, zeitliche oder räumliche Zusammenhänge innerhalb der Daten zu berücksichtigen [10, S. 2–6]. Auch im Reinforcement Learning haben Transformer-Modelle in den vergangenen Jahren zunehmend an Bedeutung gewonnen. Parisotto et al. entwickelten mit Gated Transformer-XL (GTrXL) eine speziell angepasste Transformer-Architektur, die stabile Lernprozesse bei sequenziellen Entscheidungsaufgaben ermöglicht und die Anwendung von Transformern im Reinforcement Learning erheblich voranbrachte [11]. Aufbauend auf diesen Erfolgen zeigten Chen et al. mit dem Decision Transformer, dass Reinforcement-Learning-Probleme als reine Sequenzmodellierungsaufgaben formuliert werden können. Anstatt eine klassische Wertfunktion oder Policy zu lernen, erzeugt das Modell Aktionen auf Basis vergangener Zustände, Aktionen und angestrebter Returns und nutzt dabei die Stärken der Transformer-Architektur für die Entscheidungsfindung [12]. Durch ihre Fähigkeit, langfristige Abhängigkeiten effizient zu modellieren und Sequenzen parallel zu verarbeiten, stellen

Transformer mittlerweile eine vielversprechende Alternative zu rekurrenten Architekturen wie LSTM dar und werden zunehmend auch für komplexe Aufgaben im Reinforcement Learning eingesetzt.

2.5. Wahrnehmung in RL-Agenten

Die Qualität der Wahrnehmung bestimmt maßgeblich, welche Informationen dem Agenten zur Entscheidungsfindung zur Verfügung stehen. In Unity ML-Agents werden Beobachtungen als Eingabevektoren an das neuronale Netz übergeben. Je nach Sensortyp unterscheiden sich Informationsdichte, Rechenaufwand und die Anforderungen an die Netzwerkarchitektur erheblich.

2.5.1. Vektorbeobachtungen (VectorSensor)

Die einfachste Form der Wahrnehmung in Unity ML-Agents erfolgt über Vektorbeobachtungen (Vector Observations). Dabei werden ausgewählte Zustandsinformationen der Umgebung als numerischer Vektor direkt an das neuronale Netzwerk übergeben. Typische Beobachtungen umfassen beispielsweise die Position, Geschwindigkeit, Orientierung oder den Abstand zu relevanten Objekten. Da keine zusätzliche Merkmalsextraktion notwendig ist, sind Vektorbeobachtungen besonders recheneffizient und eignen sich vor allem für Szenarien, in denen sich der Zustand der Umgebung durch mathematische Größen vollständig beschreiben lässt [3, S. 12 f.]. **Im Rahmen dieser Arbeit erhält der Agent pro Zeitschritt insgesamt 14 numerische Beobachtungswerte. Dazu zählen die normalisierte Eigengeschwindigkeit des Agenten, ein Indikator für den Bodenkontakt (isGrounded), der Richtungsvektor zum Zielobjekt sowie die Informationen eines manuell implementierten Bodensensors. Dieser basiert auf drei nach vorne gerichteten Raycasts und liefert sowohl Informationen über die erkannte Bodenart als auch die normalisierte Distanz zum jeweils getroffenen Objekt.**

2.5.2. Ray-Sensoren (RayPerceptionSensor)

Neben Vektorbeobachtungen bietet Unity ML-Agents die Möglichkeit, die Umgebung mithilfe von Ray-Sensoren (RayPerceptionSensor) wahrzunehmen. Hierbei werden virtuelle Strahlen vom Agenten ausgesendet, die Kollisionen mit Objekten in der Umgebung erkennen. Für jeden Strahl werden Informationen über den erkannten Objekttyp sowie dessen Entfernung zum

Agenten erfasst. Dadurch entsteht eine kompakte und effiziente Repräsentation der unmittelbaren Umgebung, ohne dass eine aufwendige Bildverarbeitung erforderlich ist [3, S.12f].

In dieser Arbeit wird ein RayPerceptionSensor3D eingesetzt, der insgesamt elf Strahlen über einen Sichtwinkel von 120° aussendet. Jeder Strahl kann sechs unterschiedliche Objekttypen erkennen: Wall, Lava, Hole, Goal, Obstacle und Bridge. Die maximale Reichweite beträgt zwölf Zellen. Zusätzlich werden zwei aufeinanderfolgende Beobachtungen gestapelt (Stacked Raycasts = 2). Durch diese zeitliche Erweiterung erhält der Agent implizite Informationen über Veränderungen in seiner Umgebung und kann beispielsweise Bewegungen oder Richtungsänderungen besser erkennen.

2.5.3. Kamerasensor (CameraSensor)

BRAUCHEN WIR DEN ÜBERHAUPT AUFGÄHLEN

2.5.4. Beobachtungsraum und Normalisierung

Unabhängig vom verwendeten Sensortyp spielt die Vorverarbeitung der Beobachtungsdaten eine wichtige Rolle für die Stabilität des Lernprozesses. Damit einzelne Merkmale das Training nicht aufgrund ihrer Größenordnung dominieren, sollten sämtliche Eingabewerte in vergleichbaren Wertebereichen vorliegen [13, S.179]. Nicht normalisierte Eingaben können zu instabilen Gradienten und einer schlechteren Konvergenz des neuronalen Netzes führen. **Aus diesem Grund werden in dieser Arbeit sämtliche durch den VectorSensor bereitgestellten Beobachtungen auf das Intervall $[-1,1]$ skaliert. Die vom RayPerceptionSensor erzeugten Distanz- und Klassifikationsinformationen werden hingegen automatisch durch Unity ML-Agents normalisiert. Dadurch liegen alle Beobachtungen in einem konsistenten numerischen Bereich vor, was effizientes und stabiles Training der Agenten unterstützt.**

2.6. Unity ML-Agents Toolkit

Das Unity ML-Agents Toolkit ist ein Open-Source-Framework, das die Entwicklung und das Training von Reinforcement-Learning-Agenten innerhalb der Unity-Spielengine ermöglicht [3, S.11f]. Es verbindet die leistungsfähigen Simulations- und Visualisierungsmöglichkeiten von Unity mit modernen Verfahren des maschinellen Lernens. Hierzu wird eine Unity-basierte Simulationsumgebung mit einem Python-basierten Trainingsprozess gekoppelt, sodass Agen-

ten mithilfe von Deep-Reinforcement-Learning-Algorithmen wie beispielsweise Proximal Policy Optimization (PPO) trainiert werden können [3, S.11f].

2.6.1. Architektur

Die Architektur des ML-Agents Toolkits besteht aus zwei voneinander getrennten Komponenten: der Simulationsumgebung in Unity und dem Trainingsprozess in Python [3, S.11ff]. Auf der Unity-Seite befinden sich die Agenten und die Trainingsumgebung. Der Agent sammelt während der Interaktion Beobachtungen aus seiner Umgebung, führt Aktionen aus und erhält hierfür entsprechende Rewards. Die Umgebung simuliert dabei die Auswirkungen der Aktionen und stellt die aktuellen Zustände für den nächsten Zeitschritt bereit. Auf der Python-Seite läuft der eigentliche Trainingsprozess. Dieser empfängt die von der Unity-Simulation erzeugten Rollouts, berechnet anhand der gesammelten Erfahrungen die Gradienten des neuronalen Netzes und aktualisiert die Modellparameter. Anschließend werden die aktualisierten Gewichte zurück an die Unity-Umgebung übertragen. Die Kommunikation zwischen beiden Komponenten erfolgt über gRPC (Google Remote Procedure Call). Durch diese Trennung können Simulation und Training unabhängig voneinander ausgeführt und flexibel erweitert werden [3, S.11ff].

2.6.2. Komponenten

Für die Implementierung von RL-Agenten stellt das ML-Agents Toolkit mehrere zentrale Komponenten bereit. Die wichtigste Komponente ist die Klasse Agent, welche die grundlegende Schnittstelle zwischen Agent und Trainingsumgebung bildet. Über Methoden wie `CollectObservations()` werden Beobachtungen erfasst, während `OnActionReceived()` die vom neuronalen Netzwerk ausgegebenen Aktionen verarbeitet. Mit `OnEpisodeBegin()` kann zudem die Umgebung zu Beginn einer neuen Episode zurückgesetzt werden [14]. Die Behavior Parameters definieren die Eigenschaften eines Agenten, insbesondere den Aktionsraum und den Beobachtungsraum. Darüber hinaus legen sie fest, welches trainierte Modell beziehungsweise welches Trainingsverhalten mit dem Agenten verknüpft ist [14]. Der Decision Requester steuert, in welchen zeitlichen Abständen der Agent neue Entscheidungen anfordert. Dadurch kann festgelegt werden, ob der Agent in jedem Simulationsschritt oder nur in bestimmten Intervallen eine neue Aktion auswählt [15]. Ergänzt wird das System durch verschiedene Sensor Components, welche die Wahrnehmung der Umgebung ermöglichen. Hierzu zählen beispielsweise der VectorSensor für numerische Zustandsinformationen, der RayPerception-

Sensor für strahlbasierte Umgebungswahrnehmung sowie der CameraSensor für bildbasierte Beobachtungen [16].

2.6.3. Trainings-Workflow

Der Trainingsprozess wird über eine YAML-Konfigurationsdatei gesteuert. In dieser werden unter anderem der verwendete Lernalgorithmus, Hyperparameter, Netzwerkarchitektur und Trainingsdauer definiert. Dadurch kann das Verhalten des Trainingsprozesses flexibel an unterschiedliche Anwendungsfälle angepasst werden [17]. Während des Trainings sammelt der Agent Erfahrungen innerhalb der Simulationsumgebung, die vom Python-Trainer verarbeitet werden. Nach Abschluss des Trainings wird das resultierende Modell im ONNX-Format (Open Neural Network Exchange) gespeichert, sodass es direkt in Unity für die Inferenz verwendet werden kann [18]. Zur Überwachung des Lernfortschritts können Trainingsmetriken wie Reward, Episodenlänge sowie Verluste der Policy- und Value-Netzwerke mit TensorBoard visualisiert werden [19]. Im Rahmen dieser Arbeit wird die Version 0.30.0 des Unity ML-Agents Toolkits eingesetzt. Da diese Version keine öffentliche Schnittstelle zur Implementierung eigener Policy-Architekturen bereitstellt, werden die untersuchten LSTM- und Transformer-Modelle durch Anpassungen der Python-Trainingsumgebung integriert. Dadurch können neben den standardmäßig verfügbaren Netzwerkarchitekturen auch sequenzielle Gedächtnismodelle evaluiert werden.

3. Stand der Technik

Nachdem in Kapitel 2 die theoretischen Grundlagen der eingesetzten Verfahren erläutert wurden, ordnet dieses Kapitel die vorliegende Arbeit in den aktuellen Forschungsstand ein. Zunächst werden etablierte Simulationsplattformen für das Deep Reinforcement Learning verglichen und die Wahl des Unity-ML-Agents-Frameworks begründet. Anschließend wird der Forschungsstand zur lernbasierten Navigation in dreidimensionalen Umgebungen betrachtet – vom Umgang mit partieller Beobachtbarkeit über die Belohnungsgestaltung bis zur Generalisierung auf unbekannte Umgebungen. Abschließend wird aus dem dargestellten Forschungsstand der Handlungsbedarf abgeleitet, an dem die vorliegende Arbeit ansetzt.

3.1. Simulationsumgebungen und das Unity-ML-Agents-Framework

Verfahren des Deep Reinforcement Learning benötigen typischerweise Millionen von Interaktionsschritten, bis sich ein brauchbares Verhalten einstellt. Das Training findet daher fast ausschließlich in simulierten Umgebungen statt, in denen die Simulationsgeschwindigkeit gegenüber der Echtzeit erhöht und Episoden beliebig oft wiederholt werden können. In der Forschung haben sich hierfür verschiedene Plattformen etabliert. DeepMind Lab stellt auf Basis der Quake-III-Engine prozedural erzeugbare 3D-Labyrinth bereit und wurde gezielt für die Navigations- und Explorationsforschung entwickelt. [20] Einen ähnlichen Ansatz verfolgt ViZDoom auf Basis der Doom-Engine, das RL-Forschung mit rein visuellen Beobachtungen ermöglicht. [21] Beide Plattformen sind jedoch auf ihre jeweilige Engine festgelegt und erlauben nur begrenzte Anpassungen der Weltlogik.

Das Unity-ML-Agents-Toolkit verfolgt demgegenüber einen allgemeineren Ansatz: Es verbindet die Unity-Engine als frei gestaltbare Simulationsumgebung mit einer Python-Schnittstelle für das Training. Juliani et al. [3] argumentieren, dass Spiele-Engines wie Unity durch ihre flexible Physiksimulation, die visuelle Gestaltungsfreiheit und die Steuerbarkeit der Simulationsgeschwindigkeit besonders geeignete Plattformen für die Entwicklung intelligenter Agenten darstellen. Für die vorliegende Arbeit ist darüber hinaus entscheidend, dass die Weltlogik — Zelltypen, Gefahrenfelder, prozedurale Layouterzeugung — vollständig selbst gestaltbar ist, was bei den engine-gebundenen Plattformen nur eingeschränkt möglich wäre.

3.2. Lernbasierte Navigation in 3D-Umgebungen

Die autonome Navigation in dreidimensionalen Umgebungen zählt zu den klassischen Anwendungsfeldern des Deep Reinforcement Learning. Im Hinblick auf die Zielsetzung dieser Arbeit — einen Agenten zu entwickeln, der generalisierbares Navigationsverhalten in Labyrinthen mit Gefahrenfeldern erlernt — ist es unabdingbar, den Forschungsstand zur lernbasierten Navigation, zum Umgang mit partieller Beobachtbarkeit, zur Belohnungsgestaltung und zur Generalisierung näher zu betrachten. In den folgenden Teilkapiteln wird der Forschungsstand hierzu erläutert.

3.2.1. Anfänge der lernbasierten Navigationsforschung

Die moderne Erforschung der lernbasierten Navigation in 3D-Umgebungen nahm ihren Ausgang mit dem Erfolg des Deep Reinforcement Learning auf zweidimensionalen Spielum-

gebungen durch Mnih et al. [22] im Jahr 2015. In der Folge verlagerte sich der Forschungsschwerpunkt auf komplexere dreidimensionale Welten.

Erstmals untersuchten Mirowski et al. [23] im Jahr 2017 systematisch das Navigationslernen in großen 3D-Labyrinthen allein auf Basis visueller Beobachtungen. Sie erkannten, dass reine End-to-End-Navigation aus Rohbeobachtungen äußerst datenhungrig ist, und führten zusätzliche Hilfsaufgaben (engl. Auxiliary Tasks) ein, darunter die Vorhersage von Tiefeninformationen und die Erkennung bereits besuchter Orte. In ihren Untersuchungen kamen sie zum Ergebnis, dass diese Hilfsaufgaben das Navigationslernen erheblich beschleunigen, da sie dem Netz zusätzliches Struktursignal über die räumliche Beschaffenheit der Umgebung liefern. Zudem setzten sie rekurrente Netzarchitekturen ein, um Informationen über die Zeit hinweg zu aggregieren. [23]

Im selben Jahr veröffentlichten Zhu et al. [24] ihre Forschungsergebnisse zur zielgerichteten visuellen Navigation in Innenraumszenen. Sie untersuchten einen Ansatz, bei dem das anzusteuernde Ziel selbst Teil der Netzeingabe ist, sodass ein einzelnes trainiertes Modell unterschiedliche Ziele ansteuern kann, ohne für jedes Ziel neu trainiert werden zu müssen. [24] Dieses Prinzip, das Ziel nicht fest in die Policy einzubauen, sondern als variable Information bereitzustellen, findet sich auch im Aufbau der vorliegenden Arbeit wieder, in der Start- und Zielposition in jeder Episode zufällig neu platziert werden.

3.2.2. Grundlagen des Lernens unter partieller Beobachtbarkeit

Wie in sec:sequenzmodellierung erläutert, sind Labyrinthumgebungen partiell beobachtbar: Der Agent nimmt in jedem Zeitschritt nur einen lokalen Ausschnitt der Welt wahr. Eine rein reaktive Policy, die ausschließlich die aktuelle Beobachtung verarbeitet, erreicht in solchen Umgebungen schnell eine Leistungsgrenze, da sie mehrdeutige Situationen nicht auflösen kann. Typische Fehlerbilder sind das Oszillieren in Sackgassen oder das wiederholte Absuchen bereits erkundeter Bereiche. [25]

Hausknecht und Stone [25] zeigten bereits 2015, dass die Erweiterung eines tiefen Q-Netzes um eine rekurrente LSTM-Schicht die Leistung in partiell beobachtbaren Aufgaben deutlich verbessert. Das LSTM verdichtet die Historie vergangener Beobachtungen in einem internen Zustandsvektor und stellt dem Agenten damit ein implizites Gedächtnis zur Verfügung. [25] Auch Mirowski et al. [23] setzten für ihre Labyrinthnavigation auf gestapelte rekurrente Einheiten. Rekurrente Gedächtnisarchitekturen stellen damit den etablierten Ausgangspunkt

für Navigationsaufgaben unter partieller Beobachtbarkeit dar; in ML-Agents sind sie in Form eines optionalen LSTM-Speichers direkt in die PPO-Trainingspipeline integriert. [3]

Rekurrente Netze weisen jedoch einen strukturellen Nachteil auf: Sämtliche Informationen vergangener Zeitschritte müssen durch einen komprimierten internen Zustand fester Größe propagiert werden. Weit zurückliegende Beobachtungen können dabei verblassen, und das Training über lange Sequenzen ist schwer parallelisierbar. Diese Einschränkungen bilden die wesentliche Motivation für den Einsatz von Aufmerksamkeitsmechanismen, deren Grundlagen in sec:transformer-grundlagen erläutert wurden. [11]

3.2.3. Grundlagen der Belohnungsgestaltung

Neben dem Lernalgorithmus und der Modellarchitektur hat die Gestaltung der Belohnungsfunktion wesentlichen Einfluss auf den Trainingserfolg. In Navigationsaufgaben ist die natürliche Belohnung – das Erreichen des Ziels – spärlich (engl. sparse), da sie erst am Ende einer erfolgreichen Episode auftritt. Ein zufällig explorierender Agent findet das Ziel in großen Labyrinthen anfangs nur selten, wodurch das Lernsignal weitgehend ausbleibt. [2]

Als Reward Shaping wird die Anreicherung der Belohnungsfunktion um Zwischenbelohnungen bezeichnet, etwa Strafen für Zeitverbrauch oder für das Betreten gefährlicher Felder. Ng et al. [26] zeigten bereits 1999, dass potentialbasiertes Reward Shaping die optimale Policy nicht verändert, während naiv gewählte Zwischenbelohnungen unerwünschtes Verhalten hervorrufen können, beispielsweise das wiederholte Einsammeln derselben Teilbelohnung anstelle der eigentlichen Zielerreichung. [26] Für die vorliegende Arbeit ergibt sich daraus die Anforderung, die Belohnungsfunktion nicht nur qualitativ zu beschreiben, sondern mit konkreten, begründeten Zahlenwerten zu spezifizieren. Ein verbreitetes Grundmuster für Navigationsaufgaben ist die Kombination aus einer positiven Terminalbelohnung für das Erreichen des Ziels, negativen Terminalbelohnungen für das Scheitern sowie einer kleinen negativen Belohnung pro Zeitschritt, die als Zeitdruck wirkt und passives Verhalten verhindert. [2]

3.2.4. Generalisierung auf unbekannte Umgebungen

Ein zentrales Qualitätskriterium lernender Navigationsagenten ist, dass sie nicht einzelne Karten auswendig lernen, sondern übertragbares Verhalten entwickeln. Dieses Problem ist in der Forschung als Generalisierungsproblem bekannt. Zhang et al. [27] untersuchten 2018 das Überanpassungsverhalten von Deep-RL-Agenten und kamen zum Ergebnis, dass Agenten

bei Training auf einer festen, kleinen Menge von Umgebungen massiv überanpassen: Ihre Leistung bricht auf strukturell ähnlichen, aber ungesesehenen Varianten drastisch ein, obwohl die Trainingsleistung hoch ist. [27]

Cobbe et al. [28] quantifizierten diesen Effekt ein Jahr später systematisch anhand prozedural generierter Level. Sie wiesen nach, dass überraschend große Trainingsmengen, in ihren Experimenten mehrere Tausend Levelvarianten, erforderlich sein können, bevor die Leistung auf ungesesehenen Leveln mit der Trainingsleistung gleichzieht. [28] Als Konsequenz etablierten Cobbe et al. [29] mit dem Procgen Benchmark eine Evaluationsumgebung, in der Trainings- und Testlevel strikt getrennt prozedural erzeugt werden, um tatsächliche Generalisierung anstelle von Memorierung zu messen. [29]

Aus diesen Arbeiten lassen sich zwei methodische Anforderungen für die vorliegende Arbeit ableiten. Erstens muss die Trainingsumgebung ausreichende Variation bereitstellen. Das in dieser Arbeit entwickelte Map-System mit mehreren Grundlayouts, prozeduraler Layouterzeugung sowie pro Episode zufällig platzierten Start-, Ziel- und Hindernispositionen dient genau diesem Zweck. Zweitens muss die Evaluation auf zurückgehaltenen Karten erfolgen, die während des Trainings nicht verwendet wurden, da Erfolgsraten auf Trainingskarten die tatsächliche Fähigkeit des Agenten systematisch überschätzen würden. [27], [28]

3.3. Ableitung des Handlungsbedarfs für die vorliegende Arbeit

Der dargestellte Stand der Technik zeigt, dass die einzelnen Bausteine der vorliegenden Arbeit jeweils gut erforscht sind: PPO ist ein etabliertes, robustes Trainingsverfahren [7], Unity ML-Agents eine erprobte Plattform für selbst gestaltete Trainingsumgebungen [3], die Labyrinthnavigation ein klassisches Untersuchungsfeld [23] und die Notwendigkeit von Umgebungsvariation für die Generalisierung empirisch belegt [27], [28]. Der Einsatz von Transformern als Entscheidungsmodell im Online-Reinforcement-Learning ist durch den GTrXL prinzipiell demonstriert [11], allerdings überwiegend auf großskaligen Forschungsplattformen mit erheblichem Rechenaufwand.

4. Systemarchitektur und Umgebung

Dieses Kapitel beschreibt das entwickelte System in seinem Endzustand als Versuchsapparat: Dargestellt wird, was das Experiment voraussetzt, um die Forschungsfragen F1-F3 beantworten zu können. Die Umgebung steht dabei bewusst vor der Methodik, denn das Vergleichsdesign (Kapitel 5) ist nur vor dem Hintergrund der Welt verständlich, in der gemessen wird. Wie das System entstanden ist, einschließlich der revidierten Zwischenstände, dokumentiert Kapitel 7 (Abschnitt 7.2.6.). Dieses Kapitel liefert den Zustand, Kapitel 7 die Genese.

4.1. Gesamtüberblick

Das Gesamtsystem bildet einen geschlossenen Regelkreis aus Unity-Umgebung, PPO-Trainer und TensorBoard-Monitoring: Die Unity-Szene liefert Beobachtungen und Belohnungssignale, der Python-Trainer optimiert die Policy und sendet Aktionen zurück, TensorBoard protokolliert den Trainingsverlauf. Alle drei verglichenen Architekturen laufen unter identischer Kopplung in demselben Apparat. Damit ist die Netzarchitektur die einzige zwischen den Vergleichsgruppen variierte Größe. Das darauf aufbauende Vergleichsdesign wird in Kapitel 5 entwickelt.

Die Umgebung diene als reproduzierbarer Testbereich über Seeds, in dem die Forschungsfragen F1-F3 untersucht werden konnten: Jedes Kartendesign konnte deterministisch aus einem Seed generiert werden und konnte somit vor Beginn des Trainings fixiert, frei wiederverwendet oder absichtlich vom Training ausgeschlossen werden (Abschnitt 4.2.3.). Die verwendeten Versionen und die Struktur des Repositories sind in Tabelle 2 zusammengefasst.

Komponente	Spezifikation
Engine	Unity 6 (6000.2.6f1)
ML-Agents (Unity-Paket)	com.unity.ml-agents 2.0.2
ML-Agents (Python-Trainer)	mlagents 0.30.0 (Details: Tabelle 12)
Kopplung Unity-Trainer	ML-Agents-gRPC; je --num-envs-Instanz ein eigener Headless-Unity-Prozess mit eigenem Port
Assets/Scripts/	Laufzeitcode: Map, Agent, Camera, Audio, Competition, Visual
Assets/Editor/	Offline-Werkzeuge: Generatoren, Szenen-Builder, Validatoren
Assets/Scenes/, Assets/Prefabs/	Szenen und Prefabs
training/	Patch-Skript, Custom-Policies, Export, Reports
config/	Trainer-Konfigurationen (YAML)
results/	Trainingsläufe, Checkpoints, ONNX-Exporte

Tabelle 2 – Software-Stack und Repository-Struktur des Gesamtsystems.

4.2. Map-System

4.2.1. Datenmodell

Eine Karte besteht aus typisierten Zellen, die in einem zweidimensionalen System angeordnet sind (Tabelle 3). Mehrere Aspekte dieses Datenmodells sind relevant für die Beantwortung der Forschungsfragen. Erstens schafft die Menge der Zelltypen mit Lava und Loch in Verbindung mit den Sackgassen der prozeduralen Layouts eine größere Vielfalt der Teil-Observabilität, mit der das temporäre Gedächtnis umgehen muss (F1, siehe die Problemstellung in Abschnitt 1.4.). Zweitens verbindet der Zelltyp Plattform den Inhalt der Karte mit dem Konzept der Lösbarkeit: Lavas können nur mit einer Plattform oder durch Springen aus Tiefe 1 überwunden werden. Diese Logik rechtfertigt das in Abschnitt 4.2.3. erklärte Konzept der Lösbarkeit und stellt die Quelle der Fehler in der Generalisierungsmetrik dar (F3).

Element	Spezifikation
CellType (Enum, 9 Werte)	Empty, Floor, Wall, Obstacle, Goal, SpawnPoint, Lava, Hole, Platform
MapData (ScriptableObject)	width, height, CellType[] cells; Zellindex $y * width + x$; Zugriff über GetCell/SetCell
Laufzeitfelder	cellHeightOffsets (Default 0.75); Flag noRuntimeObstacles

Tabelle 3 – Datenmodell der Karten: Zelltyp-Satz und Felder von MapData.

4.2.2. MapGenerator zur Laufzeit

Zur Laufzeit instanziiert der MapGenerator nur ein bestimmtes Layout als 3D-Szene. Er fungiert nur als Renderer. Wie im Offline-Pipeline (Abschnitt 4.2.3.) detailliert beschrieben, erfolgen die Hinderniskonstruktion und die Überprüfung der Lösbarkeit außerhalb des MapGenerators. Aufgrund dieser Trennung wird es möglich, Karten zu generieren (F3, Abschnitt 5.5.). Durch die Trennung ist es plausibel, Karten vor Beginn des Trainings zu erstellen, diese „einzufrieren“ und einige als ungesehenes Bewertungssets vorzubehalten. Es ist unmöglich zu garantieren, dass eine Testkarte, während der Laufzeit der Kartenübertragung, im Trainingsprozess nicht präsentiert wurde, weshalb diese Trennung eine notwendige Bedingung ist, um Generalisierung zu messen.

Das Curriculum ist als eigener Training Mode implementiert und nicht als Karten-Auswahlmodus (Tabelle 4). Die gestufte Schwierigkeit ist bewusst als kontrollierte Trainingsbedingung konzipiert und entsteht nicht zufällig durch die Kartenauswahl. Die methodische Begründung folgt in Kapitel 5, die technische Umsetzung in Kapitel 7.

Für den Architekturvergleich ist außerdem der Multi-Area-Betrieb entscheidend. Mehrere identische Trainingsflächen pro Szene gewährleisten für jede Architektur denselben Sampling-Durchsatz und schaffen damit die Voraussetzung für einen fairen Vergleich. Kapitel 5 erläutert das Versuchsdesign. Die Vergleichs-Szene, in der alle drei Architekturen gleichzeitig mit derselben Anzahl an Trainingsflächen trainieren, bildet dabei die Grundlage dieser Aussage. Die Anzahl der Trainingsflächen pro Szene ist in Tabelle 4 dokumentiert.

Element	Spezifikation
Tile-Pool	wiederverwendete Kacheln mit Prefab-Mapping je Zelltyp; kein Instantiate/Destroy je Episode
Spawn- und Zielposition	im Trainings-Prefab vordefiniert aus den gebackenen Layout-Zellen (genau je eine pro prozeduralem Layout); die dynamische Meide-Logik (Lava-/Hole-Nachbarn, Ziel $\neq$ Spawn) ist nur Standard-/Fallback-Pfad; Ziel-Variation entsteht offline über den Layout-Pool (Tabelle 6), nicht zur Laufzeit; Marker um +0.5 auf der Y-Achse versetzt; optionales Kamera-Framing
KillZone	persistente Trigger-Box bei $y = -20$; terminiert Episoden nach Sturz in ein Loch (Terminierungsmechanik; Entstehung: Kapitel 7)
Platzierungs-Modi	Spawn-, Goal- und ObstaclePlacementMode; MapSelectionMode {Fixed, Random, Sequential}; Curriculum-Quelle: CurriculumTracker.GetNextLayout()
Trainingsflächen je Szene	Training_MultiArea: 10; Transformer_Test_V2: 16; Vergleichs-Szene „Training Area“: 27 (3×9 , d. h. 9 je Architektur); Einzelszenen: 1 je Prozess, skaliert über --num-envs

Tabelle 4 – Laufzeit-Kartenaufbau: Objektverwaltung, Platzierungslogik und Trainingsflächen je Szene.

4.2.3. Prozedurale Generierung als Umsetzung der Messbarkeitsbedingung

Kapitel 1 formuliert die zentrale Bedingung für den Nachweis von Generalisierung (Abschnitt 1.3.): Generalisierung kann nur auf Layouts nachgewiesen werden, die während des Trainings nie gesehen wurden. Dafür sind systematisch erzeugbare Layouts sowie ein zurückgehaltenes, ungesehenes Test-Set erforderlich. Die prozedurale Generierung schafft diese Voraussetzung. Sie ist eine grundlegende Voraussetzung für den Nachweis der Generalisierung (F3).

Die Layouts entstehen in einer graphbasierten Pipeline aus Räumen und Korridoren (Tabelle 5). Jede erzeugte Karte durchläuft abschließend eine Lösbarkeitsprüfung: Der SemanticPathfinder verifiziert per Breitensuche über die 4er-Nachbarschaft, dass vom Startpunkt ein begehbare Pfad zum Ziel existiert. Welche Zelltypen dabei als begehbare gelten, folgt exakt der Hindernis-Semantik des Datenmodells (Abschnitt 4.2.1.; Regeln in Tabelle 5). Diese Garantie

ist für die Interpretation aller späteren Ergebnisse entscheidend: Scheitert ein Agent an einer Karte, ist das Scheitern dem Agenten zuzuschreiben, nicht einer unlösbaren Karte (F3).

Jedes Layout geht deterministisch aus einem Seed hervor. So lassen sich Karten systematisch und reproduzierbar erzeugen. Darauf beruht das zurückgehaltene, im Training nie gesehene Evaluations-Set. Dadurch wird der Overfitting-Index als Differenz aus Trainings- und Generalisierungs-Erfolgsrate messbar (Definition in Abschnitt 5.5., Ergebnisse in Abschnitt 8.3.).

Die erzeugten Layouts sind zudem nach Schwierigkeit gestuft (Tabelle 6). Diese aufsteigende Schwierigkeitsleiter definiert die Progression des Curriculum-Trainings, deren Wirkung auf die Konvergenz der Architekturen später gemessen wird (F1/F2, Abschnitt 8.2.). Die zugehörige Trainingsbedingung legt Kapitel 5 fest, ihre Entstehung schildert Kapitel 7.

Schließlich erzeugt die Hindernis-Semantik der Pipeline gezielt Situationen, die für die Forschungsfragen entscheidend sind: Terminal-Korridore enden blind in einem Loch, Lava bildet Sprung-Hürden und Verzweigungen führen in Sackgassen. Gerade diese partiell beobachtbaren Situationen erfordern temporales Gedächtnis (F1). Die Schwierigkeit der Umgebung entsteht damit nicht zufällig, sondern ist gezielt konstruiert.

Stufe	Spezifikation
Einstiegspunkt	<code>ProceduralLayoutGenerator.GenerateLayout(seed, difficulty)</code> ; bis zu 10 Generierungsversuche je Aufruf; Aufruf ausschließlich in Editor-Skripten (<code>MapGeneratorEditor</code> , <code>CurriculumV2Builder</code>), nicht zur Laufzeit
Pipeline	<code>BuildTopology</code> → Räume → Korridore → Wände → Spawn/Ziel → Coverage $\geq 15\%$ → Hindernis-Cluster → Plattformen → Pfad-Check
RoomCorridorGraph	<code>BORDER = 2</code> ; <code>MIN_CORRIDOR_LEN = 4</code> ; Korridore 2 Kacheln breit; Raumtypen Start, Goal, DeadEnd; Goal-Raum ist der Knoten mit größter Manhattan-Distanz (zu 75 %), mit 25 % Wahrscheinlichkeit der zweitweiteste (<code>goalIndex = 1</code> bei <code>rng < 0.25</code>); optionale Loops
ObstacleClusterPlacer	je Korridorotyp: Goal-Korridor Lava der Tiefe 1/2/3 (Plattform ab Tiefe > 1); DeadEnd-/Terminal-Korridor Hole der Tiefe 2; Loop-Korridor zu 50 % Lava der Tiefe 1; DeadEnd-Korridor wahlweise kein Hindernis, Hole oder Lava
SemanticPathfinder	Breitensuche (BFS) über die 4er-Nachbarschaft; begehbar: Floor, SpawnPoint, Goal, Plattform sowie <code>CellType.Obstacle</code> (Rückwärtskompatibilität zu Alt-Assets); Hole nie; Lava nur bei Tiefe 1 oder mit Plattform

Tabelle 5 – Offline-Generierungspipeline: Einstiegspunkt, Konstanten und Lösbarkeitsregeln.

Element	Spezifikation
DifficultyLevel-Enum (Reihenfolge)	Trivial → TrivialCorr → TrivialBranch → TrivialHole → TrivialHazard → Easy → Medium → Hard; zusätzlich TrivialLava
Trainiertes 8-Phasen-Curriculum (CurriculumV2Builder)	Trivial → TrivialCorr → TrivialHole → TrivialLava → TrivialHazard → Easy → Medium → Hard; TrivialBranch ist keine Phase (100 erzeugte Layouts bleiben ungenutzt); TrivialLava ist reguläre Phase 3
Phasen-Schleife	loopPhases = true: nach Hard Rücksprung auf Easy (relevant für die Interpretation der Trainings-Erfolgsrate, Abschnitt 5.5.)
Layout-Pool	100–200 Seeds je Stufe, offline erzeugt und eingefroren
Konstruktion	Trivial fix 7×7 ; übrige Trivial-Stufen direkte Konstruktion mit variabler Größe (Räume 2–4, Korridorlänge 4–9, vier Formtypen); TrivialLava aus separatem LavaMapGenerator; Easy/Medium/Hard über die Graph-Pipeline (Tabelle 5)
DifficultySettings je Stufe	Grid-Größe, Korridorzahl, Verzweigungstiefe, Hindernis-Wahrscheinlichkeiten

Tabelle 6 – Schwierigkeitsleiter und trainiertes Curriculum: Enum-Reihenfolge, Phasenfolge und Konstruktionsparameter der Layout-Pools.

4.3. Agent-System

Die gesamte Agentenlogik (Beobachtungsaufbau, Aktionsausführung und Belohnungsvergabe) liegt in der Klasse LabyrinthAgent (Assets/Scripts/Agent/LabyrinthAgent.cs) und ist für alle Architekturen identisch. Die drei Vergleichsagenten unterscheiden sich ausschließlich durch ihren Behavior-Namen und die dahinterliegende Netzarchitektur (Tabelle 7).

Behavior-Name	Rolle	Beobachtungen	DecisionPeriod
MLP_Navigator	Vergleichsagent (F1–F3)	31 Vektor + 176 Ray (Tabelle 10)	5
LSTM_Navigator	Vergleichsagent (F1–F3)	31 Vektor + 176 Ray (Tabelle 10)	5
Transformer_Navigator	Vergleichsagent (F1–F3)	31 Vektor + 176 Ray (Tabelle 10)	5

Tabelle 7 – Behaviors des Agent-Systems: die drei Vergleichsagenten.

4.3.1. Aktionsraum

Der Agent handelt über einen diskreten Aktionsraum mit drei Branches (Bewegung, Drehung und Sprung) (Tabelle 8). Der Aktionsraum ist für MLP, LSTM und Transformer identisch und damit eine kontrollierte Variable des Vergleichs. Beobachtete Leistungsunterschiede sind somit durch die Netzarchitektur begründet. Das Vergleichsdesign, das auf dieser Konstanz aufbaut, wird in Kapitel 5 entwickelt.

Branch	Größe	Semantik
0 (Bewegung)	3	keine Bewegung / vorwärts / rückwärts
1 (Drehung)	3	keine Drehung / links / rechts
2 (Sprung)	3	effektiv binär (nur Wert 1 belegt); einzelner AddForce-Impuls, ausgeführt nur bei Bodenkontakt

Tabelle 8 – Diskreter Aktionsraum der drei Vergleichsagenten (drei Branches der Größe 3).

4.3.2. Observation-Space

Alle drei Vergleichsagenten erhalten denselben Beobachtungssatz: 31 Vektor-Beobachtungen und 176 Ray-Beobachtungen. Der Szenen-Builder weist MLP_Navigator, LSTM_Navigator und Transformer_Navigator über dasselbe Prefab identische Eingaben zu. Unterschiede in F1 und F2 spiegeln damit ausschließlich Unterschiede der Netzarchitektur wider. Kapitel 5 entwickelt das Vergleichsdesign auf dieser Grundlage.

Die vollständige Spezifikation des Beobachtungssatzes enthält Tabelle 10 (Abschnitt 4.4.). Für alle Agenten gilt außerdem `NumStackedVectorObservations = 1`, sodass die Vektor-Beobachtungen nicht zusätzlich gestapelt werden.

4.3.3. Bewegungs- und Sprungphysik

Der Agent bewegt sich physikbasiert über einen RigidBody, die Kinematik-Parameter (Tabelle 9) sind für alle drei Architekturen identisch. Zwei dieser Parameter sind unmittelbar experimentell relevant. moveSpeed normiert die Geschwindigkeits-Beobachtung (Tabelle 10), während die Sprungkraft die Lösbarkeitssemantik der Karten bestimmt, indem sie Lava der Tiefe 1 per Sprung überwindbar macht (Abschnitt 4.2.3.). Im Endzustand umfasst die Sprungphysik außerdem einen Wall-Climb-Guard sowie eine Begrenzung der vertikalen Geschwindigkeit (Werte in Tabelle 9). Die Hintergründe beider Mechanismen werden in der Iterationsdarstellung in Kapitel 7 erläutert (Abschnitt 7.2.6.).

Parameter	Wert	Anmerkung
turnSpeed	180°/s	
moveSpeed	5	Bezugsgröße der Geschwindigkeits-Normierung (Tabelle 10)
jumpForce	10.5	effektiver Wert aus Agent.prefab; Impuls nur bei Bodenkontakt
maxUpwardVelocity	8	effektiver Wert aus Agent.prefab; begrenzt die vertikale Geschwindigkeit
wallClimbMaxY	5	Wall-Climb-Guard (Abschnitt 7.2.6.)
Masse	1	Rotation um X- und Z-Achse fixiert (FreezeRotation)
GroundCheck	Raycast abwärts	trifft Floor, Bridge, Platform, Goal; Bedingung für den Sprung

Tabelle 9 – Bewegungs- und Sprungphysik des Agenten. jumpForce und maxUpwardVelocity sind die effektiven Werte aus Agent.prefab (Script-Defaults abweichend).

4.4. Sensorik — und die begründete Wahl der Ray-Wahrnehmung

Die Beschränkung auf Ray-basierte Wahrnehmung ist methodisch begründet. Erstens isoliert die einheitliche Ray-Sensorik aller drei Agenten die Wirkung der temporalen Architektur von der Wirkung des visuellen Encoders: Bei Kamera-Agenten wäre unklar, ob beobachtete Unterschiede aus dem Temporal-Modul oder aus dem CNN-Modul stammen. Zweitens erlaubt die niedrigdimensionale Ray-Repräsentation (Float-Vektoren statt Pixel-Arrays) deutlich kürzere Trainingszeiten und damit eine höhere Anzahl vollständiger Experiment-Wiederholungen — eine Voraussetzung statistischer Aussagekraft. Drittens liefern die typisierten Treffer der

Detectable Tags direkt interpretierbare Information darüber, welcher Hindernistyp wahrgenommen wurde; in der Generalisierungsanalyse (Abschnitt 8.3.) lässt sich damit untersuchen, woran das Verhalten auf ungesehenen Karten scheitert (F3). Der ursprünglich erwogene Sensormodalitäts-Vergleich (Kamera, Sensor-Fusion) wird als zurückgestellte Erweiterung im Ausblick (Abschnitt 10.4.) wieder aufgegriffen.

Tabelle 10 spezifiziert den vollständigen Sensor- und Beobachtungssatz der drei Vergleichsagenten. Die Tabelle ist die autoritative und einzige Quelle dieser Spezifikation; alle übrigen Stellen der Arbeit verweisen hierher, ohne Werte zu wiederholen.

Komponente	Dimension	Inhalt	Wertebereich
RayPerceptionSensor3D (horizontal)	176	11 Rays ($2 \times 5 + 1$) über 120° (MaxRayDegrees 60), Reichweite 12, SphereCast 0.25; 6 Detectable Tags (Wall, Obstacle, Lava, Hole, Goal, Bridge); je Ray Tag-Kodierung, Trefferflag und normierte Distanz; stacked = 2	[0, 1] (automatisch normalisiert)
Boden-Sensor (manuell, VectorSensor)	18	9 Raycasts abwärts (Reichweite 2), Positionen unter/vorne 1-2/diagonal/seitlich; je [Typ-Code, norm. Distanz]; Codes: Floor +1, Bridge +0.5, Hole -0.5, Lava -1, kein Treffer -1.5	Typ-Code [-1.5, 1]; Distanz [0, 1]
Eigengeschwindigkeit	3	lokale Geschwindigkeit, normiert auf move-Speed	ca. [-1, 1.6]
Bodenkontakt	1	isGrounded-Flag	{0, 1}
Zieldistanz	1	Distanz zum Ziel / maxObservationDistance (20), ungeclamped	[0, ≈ 2.25]
Zielrichtung	3	normierter Richtungsvektor zum Ziel	[-1, 1]
Wand-Raycasts	4	normierte Distanzen in vier Richtungen	[0, 1]
Line-of-Sight	1	Sichtlinien-Flag zum Ziel	{0, 1}

Tabelle 10 – Vollständige Sensor- und Beobachtungsspezifikation der drei Vergleichsagenten (autoritative Quelle): 31 Vektor-Beobachtungen plus 176 Ray-Beobachtungen.
34 von 60

4.5. Reward-System

Die Belohnungsfunktion ist für alle drei Architekturen identisch und wurde vor Beginn der Vergleichsläufe festgelegt (Tabelle 11). Sie bildet damit die gemeinsame Zielvorgabe des Experiments. F1 und F2 sollen Unterschiede der Netzarchitektur messen, nicht des Belohnungsdesigns. Das Vergleichsdesign, das auf dieser Konstanz aufbaut, wird in Kapitel 5 entwickelt. Die Herleitung der einzelnen Terme sowie ihre beobachtete Wirkung sind bewusst nicht Gegenstand dieses Kapitels. Sie gehören zur Entstehungsgeschichte des Systems und werden in Kapitel 7 behandelt. Hier wird ausschließlich die gemeinsame, im Experiment unveränderte Belohnungsfunktion beschrieben.

Vom Belohnungssystem des Agenten zu unterscheiden ist das Curiosity-Signal, denn es ist kein Bestandteil der Reward-Funktion, sondern ein trainerseitiges intrinsisches Signal. In der Vergleichskonfiguration ist Curiosity in allen drei Behaviors aktiv, die Asymmetrie zwischen den Architekturen liegt in der Signalstärke. Diese Abweichung wird in Abschnitt 5.3.2. begründet und dokumentiert (Tabelle 15).

Term	Wert	Anmerkung
Zielerreichung (goalReward)	+30	
Tod durch Lava bzw. Loch	-3	lava-/holeDeathPenalty
Timeout (timeoutPenalty)	-10	
Schritt-Malus (stepPenalty)	-0.002	je Timestep
Distanz-Shaping (PBRs)	$F = (\text{prevDist} - \gamma \cdot \text{currDist}) \cdot \text{scale}$	$\gamma = 1.0$, scale = 0.01
Lava-Sprung-Versuch	+1.5	degressiv: bei Wiederholung 1/4 bzw. 1/8 des Werts, danach 0
Lava-Überquerung	+8	
Loch-Überflug	-1	
Sichtlinie zum Ziel (Line-of-Sight)	+0.005	
Wall-Climb	-1	vgl. Abschnitt 7.2.6.
phaseMaxSteps	600 / 1200 (vier Phasen) / 1500 / 2000 / 2500	maximale Episodenlänge je Curriculum-Phase (Tabelle 6)
Vergabe	zentral im LabyrinthAgent	externe Trigger (Lava, Kill-Zone) melden ausschließlich über OnTriggerEnter

Tabelle 11 – Belohnungsfunktion der drei Vergleichsagenten – identisch für alle Architekturen und vor den Vergleichsläufen eingefroren. Die Curiosity-Stärken je Behavior sind Teil der Trainer-Konfiguration und stehen in Tabelle 15.

4.6. Trainingsinfrastruktur

Das Training läuft in einer Python-Umgebung (venv) mit ML-Agents 0.30.0, die für den Vergleich benötigte Transformer-Policy ist in dieser Version nicht enthalten und wird über ein Patch-Skript in die virtuelle Umgebung nachgerüstet. Der venv-Patch hält die Nachrüstung reproduzierbar und auf wenige, klar umrissene Eingriffe begrenzt. Begründung und Integrationsprozess dokumentiert Abschnitt 7.2..

Die Trainer-Konfigurationen unterscheiden sich systematisch zwischen den Architekturen, maßgeblich für den Vergleich ist `model_comparison_final_v3.yaml`. Abweichungen und ihre

Begründung dokumentiert Abschnitt 5.3.2.. Die Bestandteile der Trainingsinfrastruktur fasst Tabelle 12 zusammen.

Bestandteil	Spezifikation
Trainings-venv	mlagents 0.30.0
Patch	training/patch_mlagents.py: kopiert transformer_memory.py, erweitert settings.py und networks.py; start_training.py wendet den Patch automatisch an
Export-venv	separat (setup_and_export.ps1), torch 1.8.1+cpu
Historische Solo-Konfigurationen (Meilenstein 7)	labyrinth_training.yaml (PPO-Baseline: Lernrate 3e-4, Batch 512, Buffer 10 240, $\gamma = 0.99$, MLP 256×2 , normalize false, max_steps 6.4 M); labyrinth_transformer.yaml (sequence_length 16, memory_size 128, Curiosity 0.05, $\gamma = 0.997$, max_steps 60 M); labyrinth_lstm*.yaml
Vergleichslauf F1–F3 (maßgeblich)	model_comparison_final_v3.yaml: drei Behaviors in einem Lauf; identische Basisparameter in Abschnitt 5.3.1, sämtliche Abweichungen ausschließlich in Tabelle 15
Parallelisierung	Multi-Area (Tabelle 4) plus Headless-Prozesse über --num-envs; je Worker ein eigener gRPC-Port und eine eigene Curriculum-Datei

Tabelle 12 — Trainingsinfrastruktur: Umgebungen, Patch-Mechanismus und Konfigurationsebenen.

5. Methodik und experimentelles Design

5.1. Wissenschaftliche Rahmung

Frage	Metrik	Hypothese	zuständiger Vergleich	beantwortet in
F1: Mehrwert temporalen Gedächtnisses	Erfolgsrate (Definition: Abschnitt 5.5.)	H1	MLP vs. LSTM/Transformer	Abschnitt 8.2.
F2: LSTM vs. Transformer	Konvergenzgeschwindigkeit; finaler Leistungsscore (Definitionen: Abschnitt 5.5.)	H2	LSTM vs. Transformer	Abschnitt 8.2.
F3: Generalisierung auf unbekannte Maps	Overfitting-Index auf held-out Maps (Definition: Abschnitt 5.5.)	H3	alle drei Architekturen	Abschnitt 8.3.

Tabelle 13 – Operationalisierung der Forschungsfragen: Metrik, Hypothese, zuständiger Vergleich und Ergebnis-Abschnitt.

5.2. Vergleichsdesign

5.3. Kontrollierte Variablen und begründete Abweichungen

5.3.1. Identische Parameter

Das Grundprinzip des Anforderungskatalogs meint, dass sich zwischen den Agenten nur der Vergleichsgegenstand unterscheidet. Alles andere bleibt gleich. Dies ist in zwei Ebenen umgesetzt. Die erste Ebene ist die Umgebung und ist vollständig identisch, da alle drei Architekturen gleichzeitig im selben Trainingslauf trainieren. Das bedeutet, dass bei allen drei Architekturen die gleiche Szene, die gleichen Karten, das gleiche Curriculum mit exakt denselben Erfolgs-

Gattern und Episodenlimits vorliegt. Ebenfalls teilen sich alle drei die gleiche Physik und die gleiche Belohnungsfunktion. Auch das Agentenskript und der Beobachtungsraum sind identisch, da jeder Agent dieselben 31 Vektorobservations inklusive Zielrichtungsvektor und dieselbe Ray-Sensorik bekommt. Auch Unterbrechungen und Neustarts während des Trainings wirken sich gleichermaßen auf alle aus. Die zweite Ebene ist die Trainer-Konfiguration. Hier sind alle Parameter identisch, die nicht im nachfolgenden Abschnitt 5.3.2 als begründete Abweichung aufgeführt werden.

Parameter	Wert (alle drei Behaviors)
trainer_type	ppo
epsilon (PPO-Clipping)	0.2
lambda (GAE)	0.95
num_epoch	3
learning_rate_schedule	linear
hidden_units / num_layers (Encoder)	256 / 2
sequence_length (LSTM und Transformer)	16
max_steps (Trainingsbudget)	30.000.000
seed	42
Curiosity: aktiv, strength / gamma / Lernrate / Netz	0.05 / 0.99 / 3e-4 / 128 × 2
summary_freq / checkpoint_interval / keep_checkpoints	10.000 / 200.000 / 5
threaded	true
Parallelität / Zeitskalierung (num_envs / time-scale)	3 / 40

Tabelle 14 – Identische Trainer-Parameter aller drei Behaviors im finalen Vergleichslauf.

5.3.2. Notwendige YAML-Abweichungen und ihre Begründung

Die Trainer-Konfigurationen der drei Architekturen (model_comparison_final_v3.yaml) sind nicht zu 100 % identisch – und das ist methodisch korrekt. Die Abweichungen zerfallen in zwei Klassen. Konstitutiv ist einzig memory_type: Ohne diesen Parameter gäbe es keinen Vergleichsgegenstand – man kann Transformer und LSTM nicht vergleichen, ohne memory_type zu ändern; eine erzwungene 100%-Identität wäre nicht „fairer“, sondern sinnlos. Daneben stehen nicht-konstitutive Tuning-Abweichungen nach dem Prinzip „Best vs. Best“: Jede Architektur läuft mit ihrer jeweils besten bekannten Konfiguration. Tabelle 15 listet sämtliche Abweichungen; max_steps (30 M), seed (42) und die in Abschnitt 5.3.1 genannten Parameter sind für alle drei Behaviors identisch. Die vollständigen YAML-Konfigurationen sind im Anhang dokumentiert.

Parameter	MLP	LSTM	Transformer	Begründung
memory_type	—	lstm	transformer	konstitutiv: definiert den Vergleichsgegenstand
learning_rate	3e-4 (linear)	3e-4 (linear)	1e-4 (constant)	Best vs. Best: Transformer-Stabilität
batch_size	512	512	1024	Best vs. Best
buffer_size	40960	40960	81920	Best vs. Best
beta	5e-3 (linear)	5e-3 (linear)	5e-4 (constant)	Best vs. Best
time_horizon	512	512	256	Best vs. Best
gamma	0.995	0.995	0.997	Best vs. Best
normalize	false	true	false	Best vs. Best
Curiosity-Stärke	0.05	0.05	0.02	Best vs. Best; Curiosity selbst ist in allen drei Behaviors aktiv (Fairness-Kriterium R5)

Tabelle 15 – Abweichungen der Trainer-Konfigurationen zwischen den Architekturen (model_comparison_final_v3.yaml). Alle übrigen Parameter sind identisch (Abschnitt 5.3.1); vollständige YAMLS im Anhang.

Konsequenz dieser Abweichungen ist, dass kein strenger Kausalanspruch erhoben wird: Leistungsunterschiede (F1/F2) sind nicht allein der temporalen Architektur zuzuschreiben. Der Anspruch ist in der Abgrenzung (Kapitel 1) entsprechend abgeschwächt formuliert; die Abweichungen werden in Abschnitt 10.3. als Limitation geführt.

5.4. Abweichung in Tuning-Iterationen

Nachdem in den vorangegangenen Abschnitten das Vergleichsdesign und die kontrollierten Variablen beschrieben wurden, dokumentiert dieser Abschnitt die im Verlauf der Arbeit durchgeführten Tuning-Iterationen und die dabei aufgetretenen Abweichungen zwischen erwartetem und tatsächlich beobachtetem Trainingsverhalten. Beim Training von Reinforcement-Learning-Agenten sind solche Abweichungen kein Randphänomen, sondern der Regelfall: Die Wechselwirkungen zwischen Belohnungsfunktion, Beobachtungsraum, Netzarchitektur und Hyperparametern sind im Vorfeld nur begrenzt vorhersagbar, sodass sich das tatsächliche Agentenverhalten erst im Experiment zeigt. Die systematische Dokumentation dieser Abweichungen ist daher ein wesentlicher Bestandteil der wissenschaftlichen Nach-

vollziehbarkeit dieser Arbeit: Sie macht transparent, welche Entwurfsentscheidungen sich bewährt haben, welche verworfen wurden und aus welchen Gründen.

Dabei fällt auf, dass sich der Tuning-Aufwand sehr ungleich auf die drei Architekturen verteilt: der Transformer durchlief deutlich mehr Iterationen als die MLP-Baseline. Diese Asymmetrie ist nicht methodischer Nachlässigkeit geschuldet, sondern strukturell bedingt. Das MLP ist als Standardarchitektur nativ in Unity ML-Agents integriert und läuft mit den dokumentierten PPO-Standardwerten weitgehend ohne Anpassungsbedarf — es ist die am einfachsten zu implementierende der drei Architekturen.

5.4.1. Vorgehensweise und Bewertungsgrundlage

Jede Tuning-Iteration wurde nach einem einheitlichen Schema durchgeführt und bewertet. Ausgehend von einer Hypothese über eine erwartete Verbesserung wurde eine Konfigurationsänderung vorgenommen, ein Trainingslauf gestartet und dessen Verlauf anhand der über TensorBoard protokollierten Metriken analysiert. Als Bewertungsgrundlage dienten dabei der durchschnittliche kumulierte Episoden-Reward (Environment/Cumulative Reward) als Hauptindikator, dessen Standardabweichung als Maß für die Stabilität der Policy, die mittlere Episodenlänge sowie ergänzend die Verlustgrößen des Trainings (Policy Loss, Value Loss) und die Entropie der Policy als Indikator für den Explorationsgrad. Eine Abweichung liegt im Sinne dieses Abschnitts immer dann vor, wenn das beobachtete Verhalten von der zuvor formulierten Erwartung abweicht, unabhängig davon, ob die Abweichung negativ oder positiv ausfällt.

5.4.2. Iterationen der MLP-Baseline

Für die MLP-Baseline des Architekturvergleichs waren keine architekturspezifischen Hyperparameter-Iterationen erforderlich: Es wurde genau ein Trainingslauf durchgeführt, der ohne nennenswerte Komplikationen bis zum Ende des Trainingsbudgets durchlief. Die Konfiguration wurde einmalig aus den dokumentierten PPO-Standardwerten des Frameworks abgeleitet, im Rahmen des Vergleichsdesigns eingefroren und während des Laufs zu keinem Zeitpunkt verändert. Gerade dieser Befund — dass die Baseline ohne eigenes Tuning und ohne Wiederholungsläufe auskam, während der Transformer eine umfangreiche Iterationshistorie erforderte — ist der empirische Beleg für die eingangs beschriebene Asymmetrie des Tuning-Aufwands: Sie spiegelt nicht unterschiedliche Sorgfalt wider, sondern den unterschiedlichen Reifegrad der Framework-Integration der drei Architekturen.

Abweichungen im Sinne dieses Abschnitts traten bei der MLP-Baseline dementsprechend nicht auf der Ebene der Hyperparameter oder des Trainingsablaufs auf. Die Abweichungen der Baseline liegen vielmehr auf der Verhaltensebene – in den strukturellen Grenzen einer gedächtnislosen Policy, wie sie in Abschnitt 11. als Erwartung formuliert wurden. Ob und in welchem Ausmaß diese erwarteten Grenzen im Trainingsverlauf tatsächlich beobachtbar sind, wird zusammen mit den finalen Ergebniszahlen in Abschnitt 6.1. dargestellt und eingeordnet.

5.5. Evaluationsprotokoll

5.5.1. Primärmetriken

5.5.2. Generalisierungsmetriken

5.5.3. Statistische Auswertung

6. Modellarchitekturen

6.1. MLP-Baseline

Die Aufgabenstellung der vorliegenden Arbeit fordert neben dem Transformer-basierten Entscheidungsmodell explizit den Vergleich mit einer einfachen Basisvariante. Diese Rolle übernimmt die in diesem Abschnitt beschriebene MLP-Baseline: ein Agent, dessen Entscheidungsmodell aus einem einfachen Multilayer-Perzeptron ohne jede Form von Gedächtnis besteht. Im Folgenden werden zunächst die Rolle der Baseline im Untersuchungsdesign und der Aufbau des Netzes erläutert. Anschließend werden Beobachtungsraum, Aktionsraum, Belohnungsstruktur und Episodenlogik spezifiziert und abschließend die Erwartung an die Baseline formuliert. Der Trainingsverlauf und die Ergebniszahlen werden gemäß der Kapitelstruktur dieser Arbeit erst in der Evaluation dargestellt (Abschnitt 6.1.).

6.1.1. Rolle der Baseline im Untersuchungsdesign

Die MLP-Baseline erfüllt im Untersuchungsdesign dieser Arbeit zwei Funktionen. Erstens dient sie als unterer Vergleichsanker: Sie beantwortet die Frage, welche Leistung in der

entwickelten Labyrinthumgebung bereits ohne Gedächtnis erreichbar ist. Nur vor diesem Hintergrund lässt sich der Mehrwert gedächtnisbehafteter Architekturen – LSTM und Transformer – überhaupt quantifizieren. Zweitens diente die Baseline als technischer Nachweis, dass die gesamte Trainingspipeline aus Unity-Umgebung, ML-Agents-Anbindung, Belohnungslogik und Protokollierung funktionsfähig ist, bevor komplexere Modellvarianten untersucht wurden.

Damit der spätere Architekturvergleich wissenschaftlich belastbar ist, wurde vor dem Training ein Grundprinzip festgelegt: Zwischen den zu vergleichenden Agentenvarianten soll sich möglichst nur eine Variable unterscheiden – die Modellarchitektur beziehungsweise der Gedächtnistyp –, während alle übrigen Randbedingungen identisch bleiben. Hierzu wurden die Belohnungsstruktur, die Umgebungskonfiguration und die gemeinsamen Trainingsparameter vor dem ersten Vergleichslauf dokumentiert und eingefroren; die wenigen begründeten Abweichungen zwischen den Trainer-Konfigurationen sind in Abschnitt 5.3.2. dokumentiert. Die in den folgenden Teilabschnitten beschriebene Konfiguration der MLP-Baseline definiert damit zugleich die Referenzkonfiguration für alle weiteren Modellvarianten dieser Arbeit.

6.1.2. Aufbau des Multilayer-Perzeptrons

Das Multilayer-Perzeptron (MLP) ist die Grundform des vorwärtsgerichteten neuronalen Netzes. Es besteht aus einer Eingabeschicht, einer oder mehreren versteckten Schichten und einer Ausgabeschicht, wobei jede Schicht eine affine Transformation ihrer Eingabe mit anschließender nichtlinearer Aktivierungsfunktion berechnet. [30]

Entscheidend für die Rolle als Baseline ist eine strukturelle Eigenschaft des MLP: Es verarbeitet ausschließlich die Beobachtung des aktuellen Zeitschritts. Es existiert kein interner Zustand, der Informationen über vergangene Beobachtungen speichert – die Policy ist rein reaktiv. Beobachtet der Agent in zwei unterschiedlichen Situationen denselben Beobachtungsvektor, wählt er zwangsläufig dieselbe Aktionsverteilung, unabhängig davon, wie er in diese Situationen gelangt ist. [30]

In der vorliegenden Arbeit wird das MLP in der Standardkonfiguration des ML-Agents-Frameworks verwendet: Der Beobachtungsvektor wird durch zwei versteckte Schichten mit jeweils 256 Neuronen verarbeitet. Auf diesen gemeinsamen Netzrumpf setzen im PPO-Training zwei Ausgabeköpfe auf: die Policy, welche die Wahrscheinlichkeitsverteilungen über die diskreten Aktionen ausgibt, und der Wertschätzer (Critic), der den erwarteten Gesamtertrag des aktuellen Zustands schätzt. [3]

6.1.3. Erwartung an die Baseline

Aus der Spezifikation lässt sich die Erwartung an die Baseline unmittelbar ableiten. Dank der Zielrichtungs-Observation und der Ray-Sensorik sollte das MLP in der Lage sein, eine reaktive Zielansteuerung mit lokaler Gefahrenvermeidung zu erlernen und die Navigationsaufgabe in der Mehrzahl der Episoden zu lösen. Zugleich markiert die Gedächtnislosigkeit eine strukturelle Obergrenze: In Situationen, deren korrekte Behandlung Wissen über die eigene Vorgeschichte erfordert – insbesondere das Erkennen bereits erkundeter Bereiche und das systematische Verlassen von Sackgassen –, kann eine rein reaktive Policy prinzipbedingt keine optimale Entscheidung treffen [25]. Die Baseline wird daher erwartungsgemäß unterhalb der theoretisch möglichen Leistung sättigen.

6.2. Transformer-Memory (Custom Policy)

6.2.1. Motivation: Memory unter partieller Beobachtbarkeit

6.2.2. Eingabeverarbeitung: Beobachtungssequenz und Embedding

6.2.3. Kern der Architektur: kausaler Self-Attention-Block

6.2.4. Sequenzaggregation und Actor-Critic-Anbindung

6.3. LSTM-Memory (Custom Policy)

Der LSTM-Agent ist der erste der beiden gedächtnisbehafteten Vergleichskandidaten. Er erweitert die in Abschnitt 6.1 beschriebene Referenzkonfiguration um genau eine Komponente, nämlich um ein Gedächtnismodul zwischen Beobachtungs-Encoder und Ausgabeköpfen. Beobachtungsraum, Aktionsraum, Belohnungsstruktur und Episodenlogik sind mit der Baseline identisch. Die theoretischen Grundlagen der LSTM-Zelle wurden in Abschnitt 2.4.1 eingeführt. Im Folgenden werden die Rolle des Agenten im Untersuchungsdesign, der Aufbau und die Integrationsentscheidung einer eigenen LSTM-Implementierung sowie die Erwartung an die Architektur beschrieben.

7. Umsetzung

7.1. Aufbauphase (M1–M6)

Die Aufbauphase errichtet das grundlegende Fundament, auf dem im späteren Verlauf der Architekturvergleich aufsetzt. Ziel dieses Fundaments ist es eine kontrollierbare, prozedural generierte Labyrinth Umgebung zu schaffen. Weiter besteht das Fundament aus einem physik-basierten Agenten mit Sensorik und einem Belohnungssystem. Um das Fundament für spätere Vergleiche einsatzbereit zu machen, ist eine erste funktionsfähige Trainingspipeline ebenfalls wesentlicher Bestandteil. Die Aufbauphase lässt sich in 6 Bereiche gliedern, die in GitHub in Form von Milestones strukturiert und realisiert werden. Im folgende Abschnitt werden diese 6 Meilensteine näher betrachtet.

7.1.1. Meilenstein 1: Map-System

Den ersten Schritt macht die Umgebung. Sie bildet die Grundlage für alle weiteren Schritte. Dieser initiale Meilenstein etabliert eine erste Version und damit die Basis des Kartensystems. Beim Start der Szene erzeugt der MapGenerator eine Karte aus einem deklarativen Datenmodell (MapData), in dem durch ein zweidimensionales Zellraster für jede Zelle ein entsprechender Typ beschrieben wird. Zu den möglichen Typen gehören Wände, begehbare Boden und vorläufige Platzhalter für die in Meilenstein 4 implementierten Gefahrenelemente. Aus dieser Beschreibung wird zur Laufzeit die eigentliche 3D-Geometrie aufgebaut. Die Trennung von Kartenbeschreibung und Karteninstanziierung ist eine zentrale Designentscheidung, welche aus mehreren Gründen getroffen wurde: Karten werden durch diese Trennung versionierbar, austauschbar und prozedural erzeugbar gemacht. Weiterhin gehören zum Grundsystem der Umgebung die Platzierung von Spawn- und Zielelementen, ein Laufzeit-Reset mit Testtrigger als Vorgriff die Episodenlogik des Trainings, sowie erste Objekt-Tags (unter anderem für das Objekt Wand, Boden und Ziel). Diese sind elementarwichtig für die spätere Tag-basierte Sensorik des Agenten.

7.1.2. Meilenstein 2: Mehrere Layouts

Der zweite Meilenstein erweitert das Kartensystem vom Einzellayout zur Layout-Bibliothek. Diese Bibliothek bildet ein erstes Fundament für die angestrebten Generalisierungstests. Der MapGenerator verwaltet also seit dieser Implementierung einen Satz von Layout-Definitionen

und wählt daraus nach dem konfigurierbaren Modus. Hierbei lässt sich zwischen einem festen Layout, einem sequentiellen Modus und dem zufälligen Modus, welcher ohne Reihenfolge verschiedenen Layouts mit Wiederholungsvermeidung verwendet unterscheiden.

7.1.3. Meilenstein 3: Das Agent-Grundsystem

Der dritte Meilenstein bringt den Agenten in die Umgebung. Hierbei erweitert die zentrale Klasse LabyrinthAgent die Agent-Basisklasse des Unity-ML-Agents-Frameworks und definiert den diskreten Aktionsraum der Arbeit. Dieser besteht aus den drei Aktionszweigen für Vorwärts-/Rückwärtsbewegung, Links-/Rechtsdrehung und Sprung. Die Bewegung ist Rigid-body-basiert und damit physikkonsistent. Das ist eine Voraussetzung dafür, dass Sprünge über Lava eine echte motorische Teilaufgabe darstellen. Ein Ground-Check unterscheidet Boden – und Flugphasen und verhindert so Doppelspringen. Mit der Episodenlogik (Spawn-Platzierung und Reset über den MapGenerator) ist die Interaktionsschleife zwischen Agent und Umgebung geschlossen und vollständig.

7.1.4. Meilenstein 4: Hindernisse, Todeslogik und Sensorik

Der vierte Meilenstein verwandelt die bisher problemlos begehbare Karte in eine Aufgabe mit Risiko. Das in Meilenstein 1 beschriebene Zellmodell wird um die Hindernisse Lava und Loch erweitert. Getaggte Trigger dieser Hindernisse melden Kontakte über OnTriggerEnter an die Todeslogik des Agenten, welche daraufhin die Episode beendet. Für die Wahrnehmung des Agenten wird der RayPerceptionSensor des Frameworks konfiguriert. Dazu tasten Strahlenbündel die Umgebung ab und klassifizieren Treffer über die in Meilenstein 1 beschriebenen Objekt-Tags. Eine dedizierte Sensor-Testszene sichert die Erkennung von Objekten isoliert ab.

7.1.5. Meilenstein 5: Reward-System und erstes Training

Innerhalb des fünften Meilensteins werden die letzten, für ein erstes Training nötigen Schritte umgesetzt. Die Belohnungsfunktion kombiniert eine Zielprämie mit Todesstrafen für die Hindernisse Lava und Loch, sowie einer Timeout-Strafe, einer kleinen Zeitstrafe je Schritt und einem potentialbasierten Distanz-Shaping, welches Annäherungen an das Ziel kontinuierlich belohnt. Vergeben wird diese Prämie zentral im LabyrinthAgent. Mit der LabyrinthAgent.yaml entsteht die erste PPO-Trainerkonfiguration. Unmittelbar darauf folgen die ersten Trainingsläufe der MLP-Baseline, welche parallel zu Meilenstein 5 implementiert wurde.

7.1.6. Meilenstein 6: Prozedurale Generierung und Curriculum

Der sechste Meilenstein professionalisiert die Kartenerzeugung und legt somit das Fundament des Curriculum-Lernens. Ein Raum-Korridor-Graph erzeugt die Topologie (Räume, Korridore und Verzweigungen). Ein semantischer Pathfinder validiert jede erzeugte Karte auf deren Lösbarkeit. Unlösbare Kandidatenkarten werden sofort verworfen und anschließend neu generiert. Ein in diesem Meilenstein eingeführtes Schwierigkeits-Enum systematisiert die Karten in Klassen von Trivial über Easy, Medium und Hard bis zu spezialisierten Lernstufen. Ein Objekt-Pool (TilePool) häufiger Kartenwechsel möglichst gering. Da die Generierung seed-basiert und somit deterministisch reproduzierbar ist, lassen sich Kartensätze sauber trennen. Diese Trennung ist die Voraussetzung für die spätere Generalisierungsmessung. Des Weiteren entsteht innerhalb dieses Meilensteins das Curriculum-System. Die CurriculumConfig beschreibt hierbei die Phasen mit Kartenklassen und Aufstiegsschwellen. Der CurriculumTracker steuert den Phasenfortschritt zur Laufzeit. Phasenspezifische Episodenlängen begrenzen Timeout-Episoden je Schwierigkeitsgrad.

Damit ist die Aufbauphase abgeschlossen und bildet somit die Basis für die Integration und Optimierung der Gedächtnisarchitekturen, deren jeweilige Umsetzung in den nachfolgenden Abschnitten näher erläutert wird.

7.2. Transformer-Integration: von V1 bis zum lauffähigen Modell

7.2.1. Warum kein Fork: der venv-Patch-Ansatz

7.2.2. Warum kein Pre-Trained Modell

7.2.3. Problemklasse A - Framework-/Engine-Workarounds

7.2.4. Problemklasse B - Inference/Training-Konsistenz

7.2.5. Problemklasse C - Reward-/Curriculum-Pathologien

7.2.6. Iterationsübersicht V1-V22 (verdichtete Tabelle)

7.3. LSTM-Integration

Die LSTM-Integration ist das Gegenstück zur Transformer-Integration aus Abschnitt 7.2 und zeigt den Unterschied im Aufwand sehr deutlich. Der Transformer benötigte eine lange Iterationsserie, bis er lief. Das LSTM lief nach einer einzigen Korrektur-Runde. In diesem Abschnitt wird beschrieben, wie das LSTM eingebaut wurde, welche Probleme beim Start auftraten und was aus den Trainingsläufen lstm_v1 bis lstm_v7b gelernt wurde. ML-Agents bringt zwar eine eigene LSTM-Unterstützung mit, aber das Gedächtnismodul wurde dennoch selbst implementiert (LSTMMemory, siehe Abschnitt 6.2.2). Es wird über denselben venv-Patch eingebaut wie die Transformer-Policy. Der Grund dafür ist die Vergleichbarkeit, da das Modul als exaktes Spiegelbild des TransformerMemory gebaut ist. Es hat die gleichen Ein- und Ausgabeformen, verarbeitet die Trainingssequenzen auf die gleiche Weise und sitzt an derselben Stelle im Netz. Beide Gedächtnisvarianten unterscheiden sich also nur in ihrer inneren Logik, nicht in der Verdrahtung. Interessant ist der Größenunterschied. Das LSTM-Modul hat rund 82.000 Parameter, das Transformer-Modul bei gleicher Gedächtnisbreite ca. 1,07 Millionen. Der Vergleich stellt bewusst zwei Gedächtnis-Mechanismen gegenüber und keine gleich großen Netze.

Der erste Versuch (lstm_v1) hat gezeigt, dass eine einfache Integration nicht sofort fehlerfrei läuft. lstm_v1 brach direkt nach dem Start ab, ohne einen einzigen Trainingsschritt. Die Analyse fand vier Fehler in der Verdrahtung. Der Patch hatte das Modul zwar angelegt, aber im Netz nie aufgerufen. Stattdessen griff der alte Codepfad auf die leere native LSTM-Referenz

zu und stürzte ab. Dazu meldete das Modul dem Trainer die Gedächtnisgröße null, sodass gar kein Speicher angelegt wurde. Außerdem wurde die Sequenzlänge nicht gespeichert und der ONNX-Export nutzte die falsche Gedächtnisgröße, was beim Speichern der Checkpoints crashte. Alle vier Fehler gehören zur Problemklasse A aus Abschnitt 7.2.3 (Framework-/Engine-Workarounds) und wurden in einer Runde behoben. Das ist der zentrale Unterschied zum Transformer, da beim LSTM die Probleme auf der Integrationsebene blieben und keine weiteren Iterationen bezüglich Rewards und Parametern nötig waren.

Der erste vollständige Lauf lstm_v2 legte dann die Konfigurationsbasis. Fünf Parameter wurden gegenüber der vom Transformer übernommenen Startkonfiguration geändert. Die Normalisierung der Beobachtungen stabilisiert den Encoder, wenn die Kartengrößen im Curriculum wechseln. Der ca. 650.000 Steps große Lauf lieferte mehrere interessante Beobachtungen. Bei jedem Phasenwechsel bricht die Erfolgsrate ein und erholt sich danach wieder. Des weiteren wurden Modellgewichte aus dem Checkpoint geladen. Bei dem Curriculum-Fortschritt war dies aber nicht der Fall. Die Messwerte direkt nach dem Resume waren dadurch verzerrt. Die dritte Beobachtung ist die Geschwindigkeitsgrenze. Im Unity-Editor lief das Training mit rund 280 Steps pro Sekunde, was zur Entscheidung geführt hat auf Headless-Builds umzustellen.

Die weiteren Läufe lstm_v3 bis lstm_v7b bauten darauf auf. Deren Hauptaufgabe war es unter anderem die Balance zwischen Todesstrafen und Zielprämie sicherzustellen. Das Ergebnis dieser Serie ist die Konfiguration labyrinth_lstm_v7.yaml. Sie wurde mit ihrer Gedächtnisdimensionierung unverändert in die finale Vergleichskonfiguration übernommen.

7.4. MLP-Integration

Nachdem in Abschnitt 6.1. die MLP-Baseline auf konzeptioneller Ebene spezifiziert wurde, beschreibt dieser Abschnitt ihre technische Integration: die Anbindung der Unity-Umgebung an die ML-Agents-Trainingspipeline, die Umsetzung des Agenten im Code und im Unity-Inspector, den Aufbau der Trainingsszene sowie die Durchführung und Überwachung des Trainings. Der Trainingsverlauf ist in Abschnitt 5.4. dokumentiert; die abschließende Einordnung der Ergebnisse erfolgt in Abschnitt 6.1..

Vorab ist der Integrationsaufwand einzuordnen, der sich zwischen den drei Architekturen deutlich unterscheidet: Die MLP-Integration ist mit Abstand die einfachste. Das MLP ist die Standardarchitektur des ML-Agents-Frameworks und nativ in dessen Trainingspipeline

enthalten – weder die Netzarchitektur noch der Python-Trainer mussten angepasst werden. Der hier beschriebene Aufwand beschränkt sich daher auf die Anbindung von Umgebung und Agent, die ohnehin die gemeinsame Grundlage aller drei Architekturen bildet. Diese Asymmetrie ist gewollt und dokumentiert (Abschnitt 5.4.): Während das LSTM ebenfalls über eine native Framework-Konfiguration einbindbar war, erforderte der Transformer einen Patch der Trainingsumgebung mit entsprechendem Iterationsbedarf (Abschnitt 7.2.). Die MLP-Integration diente damit zugleich als Fundament, auf dem die aufwendigeren Integrationen aufsetzen konnten.

7.4.1. Systemarchitektur der ML-Agents-Anbindung

Das Training der Baseline basiert auf dem Zusammenspiel zweier Prozesse. Die Unity-Engine simuliert die Spielwelt: Der Agent bewegt sich in der Labyrinthumgebung, sammelt Beobachtungen und erhält Belohnungen gemäß der spezifizierten Struktur. Parallel dazu läuft außerhalb von Unity der Python-Trainingsprozess `mlagents-learn`, der die Beobachtungen und Belohnungen des Agenten empfängt, das neuronale Netz mit PPO aktualisiert und dem Agenten die Aktionen der aktuellen Policy zurückliefert. Beide Prozesse kommunizieren über eine lokale gRPC-Netzwerkverbindung (Port 5004) und müssen während des Trainings gleichzeitig laufen. Der Lernfortschritt wird kontinuierlich als Ereignisprotokoll geschrieben und kann parallel zum Training mit TensorBoard im Browser visualisiert werden. [3]

7.4.2. Integration des Agenten in Unity

Für die Integration wurde ein durchgängiges Strukturierungsprinzip angewandt: Die Funktionslogik des Agenten liegt vollständig im Code, während der Unity-Inspector ausschließlich der Konfiguration und Verdrahtung von Komponenten dient. Diese Trennung hält die Logik nachvollziehbar und versionierbar, während Parameter wie Sensorreichweiten oder Belohnungswerte ohne Codeänderung angepasst werden können.

Kern der Integration ist die Agentenklasse `LabyrinthAgent`, die von der Agent-Basisklasse des ML-Agents-Frameworks erbt und deren Lebenszyklus-Methoden überschreibt. [3] Beim Episodenstart (`OnEpisodeBegin`) stößt der Agent das Laden eines neuen Layouts an und wird auf die Spawn-Position versetzt. Zu jedem Entscheidungszeitpunkt sammelt `CollectObservations` die Beobachtungen. Die vom Modell gewählten Aktionen werden in `OnActionReceived` entgegengenommen, in physikbasierte Bewegung, Drehung und Sprung übersetzt und mit der Schrittstrafe belegt. Die Terminalereignisse – Zielerreichung sowie Tod durch Lava oder Loch – werden über Trigger-Collider erkannt. Die Umgebungsobjekte lösen lediglich das Trigger-

Ereignis aus, während die gesamte Belohnungsvergabe und Episodensteuerung zentral in der Agentenklasse erfolgt. Diese Zentralisierung stellt sicher, dass alle Belohnungswerte an einer einzigen, im Inspector konfigurierbaren Stelle liegen und für den Architekturvergleich eingefroren werden können.

Auf dem Agenten-Prefab sind drei Framework-Komponenten konfiguriert. Die Behavior Parameters definieren die Schnittstelle zwischen Agent und Trainer: den Verhaltensnamen MLP_Navigator, die Größe des manuellen Beobachtungsvektors (vgl. Tabelle 10) sowie den diskreten Aktionsraum mit drei Zweigen (Bewegung, Drehung, Sprung). Der RayPerceptionSensor3D ist als eigene Sensorkomponente direkt am Agenten angebracht und im Inspector konfiguriert; seine Beobachtungen werden vom Framework automatisch an das Netz übergeben. Der Decision Requester steuert die Entscheidungsfrequenz: Eine Entscheidung wird alle fünf Simulationsschritte angefordert, zwischen zwei Entscheidungen wird die zuletzt gewählte Aktion wiederholt. Diese Entscheidungsfrequenz reduziert den Trainingsaufwand, ohne die Steuerbarkeit in der grid-basierten Umgebung merklich einzuschränken, da sich der Agent zwischen zwei Entscheidungen nur einen Bruchteil einer Zelle weit bewegt. Ergänzend wurde ein Heuristik-Modus implementiert, in dem der Agent manuell über die Tastatur gesteuert werden kann. Dieser Modus diente der Validierung der gesamten Wirkkette vor dem ersten Training: Bewegung, Sprungmechanik, Sensorwerte, Trigger-Erkennung und Belohnungsvergabe konnten so unabhängig vom Lernverfahren geprüft werden. Zusätzlich wurde eine separate Sensor-Testszene aufgebaut, die alle Hindernistypen enthält und in der die Sensorparameter visuell kalibriert und validiert wurden. Diese Prüfschritte sind methodisch relevant: Fehler in Sensorik oder Belohnungslogik würden sich andernfalls erst indirekt über ausbleibenden Lernfortschritt zeigen und wären dann nur schwer von Konfigurationsproblemen des Lernverfahrens zu unterscheiden.

7.4.3. Trainingsszene und Parallelisierung

Das Training erfolgt in einer dedizierten Trainingsszene mit mehreren parallelen Trainingsbereichen (Multi-Area-Setup). Jeder Bereich enthält eine eigene Instanz der Labyrinthumgebung mit eigenem Map-Controller und eigenem Agenten; alle Agenten tragen denselben Verhaltensnamen und werden daher vom Trainer als Instanzen derselben Policy behandelt. Die gesammelten Erfahrungen aller Bereiche fließen in einen gemeinsamen Erfahrungspuffer, wodurch sich die Datensammlung gegenüber einem einzelnen Agenten entsprechend vervielfacht, ohne die Lernaufgabe zu verändern. [3] Da jeder Bereich seine Umgebung unabhängig

lädt, erhöht die Parallelisierung zugleich die Diversität der Erfahrungen innerhalb eines Puffers: Ein Policy-Update basiert stets auf Episoden aus mehreren, unterschiedlich konfigurierten Labyrinthen.

7.4.4. Trainingskonfiguration, Trainingsworkflow, Beobachtbarkeit und Inferenz

Die Trainingskonfiguration ist vollständig in einer versionierten YAML-Datei hinterlegt, die dem Trainer beim Start übergeben wird. Ein Trainingslauf wird über den Kommandozeilenaufruf des Trainers gestartet, wobei die Konfigurationsdatei und eine eindeutige Laufkennung (run-id) übergeben werden. Die Laufkennung bestimmt das Ergebnisverzeichnis, in dem sämtliche Artefakte des Laufs abgelegt werden: die verwendete Konfiguration, die TensorBoard-Ereignisprotokolle, die Laufstatistiken sowie die Modell-Checkpoints. Unterbrochene Läufe können über die Laufkennung fortgesetzt werden; ein Neustart unter derselben Kennung erfordert eine explizite Bestätigung, wodurch ein versehentliches Überschreiben von Ergebnissen verhindert wird. Diese Systematik stellt sicher, dass jeder Trainingslauf eindeutig identifizierbar und mitsamt seiner Konfiguration archiviert ist – eine Voraussetzung für die in der Aufgabenstellung geforderte Reproduzierbarkeit.

Während des Trainings wurden alle 10.000 Schritte TensorBoard-Zusammenfassungen der protokollierten Metriken geschrieben, was bei vollem Budget eine Lernkurve mit 200 Datenpunkten ergibt. Alle 200.000 Schritte wurde ein Modell-Checkpoint gespeichert, wobei die jeweils letzten fünf Checkpoints vorgehalten wurden; bei instabilem Trainingsverlauf ist damit ein Rückgriff auf frühere Modellstände möglich.

Nach Abschluss des Trainings exportiert der Trainer das finale Modell im ONNX-Format. Für die Inferenz wird das exportierte Modell im Unity-Inspector den Behavior Parameters des Agenten zugewiesen und der Verhaltensmodus auf Inferenz gestellt; der Agent handelt dann ohne laufenden Python-Prozess allein auf Basis der trainierten Policy, ausgeführt über die in Unity integrierte Inferenz-Laufzeitumgebung. Dieser Mechanismus wird sowohl für die Videodemonstrationen der finalen Läufe als auch für die Evaluation der Modellvarianten genutzt.

8. Evaluation

8.1. MLP-Baseline

8.2. Architektur-Vergleiche

8.3. Generalisierung auf held-out Maps

8.4. Statistische Auswertung

8.5. Diskussion

9. Übertragbarkeit und praktische Anwendbarkeit

9.1. Analogie Labyrinth

reale Navigationsszenarien

9.2. Hardware- und Software-Anforderungen

9.3. Bewertungsmatrix

9.4. Limitationen der Übertragbarkeit

10. Fazit und Ausblick

10.1. Zusammenfassung

10.2. Beantwortung der Forschungsfragen

10.3. Limitationen und Lessons Learned

10.4. Ausblick

TOOOOOOOODOOOOOOOOOOOOOOOOOOO:

11. Sec mlp baseline

Anhang

Literatur

- [1] J. Wirtz u. a., „Brave new world: service robots in the frontline“, *Journal of Service Management*, Bd. 29, Nr. 5, S. 907–931, Nov. 2018, doi: [10.1108/JOSM-04-2018-0119](https://doi.org/10.1108/JOSM-04-2018-0119).
- [2] R. S. Sutton und A. G. Barto, *Reinforcement learning: an introduction*, Second edition. in Adaptive computation and machine learning series. Cambridge, Massachusetts: The MIT Press, 2018.
- [3] A. Juliani u. a., „Unity: A General Platform for Intelligent Agents“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/1809.02627>
- [4] I. Goodfellow, „[deeplearningbook.org/contents/ml.html](https://www.deeplearningbook.org/contents/ml.html)“. Zugegriffen: 10. Juli 2026. [Online]. Verfügbar unter: <https://www.deeplearningbook.org/contents/ml.html>
- [5] K. V. Bhagwan und D. S. Kadam, „A Comprehensive Study on Artificial Intelligence and Machine Learning“, *Communication and Technology*, Bd. 4, Nr. 3, 2024.
- [6] Shaveta, „A review on machine learning“, *International Journal of Science and Research Archive*, Bd. 9, Nr. 1, S. 281–285, Mai 2023, doi: [10.30574/ijrsra.2023.9.1.0410](https://doi.org/10.30574/ijrsra.2023.9.1.0410).
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, und O. Klimov, „Proximal Policy Optimization Algorithms“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/1707.06347>
- [8] J. Schulman, P. Moritz, S. Levine, M. Jordan, und P. Abbeel, „High-Dimensional Continuous Control Using Generalized Advantage Estimation“. Zugegriffen: 8. Juli 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/1506.02438>
- [9] I. Goodfellow, Y. Bengio, und A. Courville, „[deeplearningbook.org/contents/rnn.html](https://www.deeplearningbook.org/contents/rnn.html)“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <https://www.deeplearningbook.org/contents/rnn.html>
- [10] A. Vaswani u. a., „Attention Is All You Need“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/1706.03762>
- [11] E. Parisotto u. a., „Stabilizing Transformers for Reinforcement Learning“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/1910.06764>
- [12] L. Chen u. a., „Decision Transformer: Reinforcement Learning via Sequence Modeling“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/2106.01345>

-
- [13] I. Goodfellow, „[deeplearningbook.org/contents/convnets.html](https://www.deeplearningbook.org/contents/convnets.html)“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <https://www.deeplearningbook.org/contents/convnets.html>
- [14] Unity, „Agents - Unity ML-Agents Toolkit“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <https://unity-technologies.github.io/ml-agents/Learning-Environment-Design-Agents/>
- [15] Unity, „Agents - Unity ML-Agents Toolkit“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <https://unity-technologies.github.io/ml-agents/Learning-Environment-Design-Agents/#decisions>
- [16] Unity, „Agents - Unity ML-Agents Toolkit“. Zugegriffen: 9. Juli 2026. [Online]. Verfügbar unter: <https://unity-technologies.github.io/ml-agents/Learning-Environment-Design-Agents/#observations-and-sensors>
- [17] C. Unity, „Training Configuration File - Unity ML-Agents Toolkit“. Zugegriffen: 10. Juli 2026. [Online]. Verfügbar unter: <https://unity-technologies.github.io/ml-agents/Training-Configuration-File/>
- [18] Unity, „Unity ML-Agents Toolkit“. Zugegriffen: 10. Juli 2026. [Online]. Verfügbar unter: <https://unity-technologies.github.io/ml-agents/>
- [19] T. Unity, „Using TensorBoard to Observe Training - Unity ML-Agents Toolkit“. Zugegriffen: 10. Juli 2026. [Online]. Verfügbar unter: <https://unity-technologies.github.io/ml-agents/Using-Tensorboard/>
- [20] C. Beattie u. a., „DeepMind Lab“. [Online]. Verfügbar unter: <http://arxiv.org/abs/1612.03801>
- [21] M. Kempka, M. Wydmuch, G. Runc, J. Toczek, und W. Jaśkowski, „ViZDoom: A Doom-based AI Research Platform for Visual Reinforcement Learning“, in *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2016, S. 1–8. doi: [10.1109/CIG.2016.7860433](https://doi.org/10.1109/CIG.2016.7860433).
- [22] V. Mnih u. a., „Human-level control through deep reinforcement learning“, *Nature*, Bd. 518, Nr. 7540, S. 529–533, 2015, doi: [10.1038/nature14236](https://doi.org/10.1038/nature14236).
- [23] P. Mirowski u. a., „Learning to Navigate in Complex Environments“, in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Verfügbar unter: <http://arxiv.org/abs/1611.03673>

-
- [24] Y. Zhu u. a., „Target-driven Visual Navigation in Indoor Scenes using Deep Reinforcement Learning“, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2017, S. 3357–3364. doi: [10.1109/ICRA.2017.7989381](https://doi.org/10.1109/ICRA.2017.7989381).
- [25] M. Hausknecht und P. Stone, „Deep Recurrent Q-Learning for Partially Observable MDPs“, in *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015. [Online]. Verfügbar unter: <http://arxiv.org/abs/1507.06527>
- [26] A. Y. Ng, D. Harada, und S. Russell, „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping“, in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, S. 278–287.
- [27] C. Zhang, O. Vinyals, R. Munos, und S. Bengio, „A Study on Overfitting in Deep Reinforcement Learning“. [Online]. Verfügbar unter: <http://arxiv.org/abs/1804.06893>
- [28] K. Cobbe, O. Klimov, C. Hesse, T. Kim, und J. Schulman, „Quantifying Generalization in Reinforcement Learning“, in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, S. 1282–1289.
- [29] K. Cobbe, C. Hesse, J. Hilton, und J. Schulman, „Leveraging Procedural Generation to Benchmark Reinforcement Learning“, in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, S. 2048–2056.
- [30] I. Goodfellow, Y. Bengio, und A. Courville, *Deep Learning*. in Adaptive computation and machine learning series. Cambridge, Massachusetts: The MIT Press, 2016.

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Studienarbeit

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Friedrichshafen, 12.07.2026

Finn Ludwig, Ekaterina Karavaeva, David Pelcz, Alexander Bernecker